

Olist Agentic Analytics Copilot

Build-and-Learn Tutorial: RAG + Knowledge Graph + MCP + LangGraph + SQL (on real e-commerce data)

Goal: Build a system that lets anyone ask plain-English questions about a real 9-table e-commerce database and get correct, validated, explained answers. Learn each technology by building the piece of the system that needs it.

How to use this tutorial

Every step follows the same pattern:

-  **File:** the exact path to create, relative to the project root.
- **Code block:** the full contents of that file.
-  **What's happening:** a plain-language explanation of the code.
-  **Why it's built this way:** the design decision and what would go wrong otherwise.
-  **Checkpoint:** a command to run that proves the step works.

Rules:

1. Always run commands from the **project root** folder.
 2. Always run Python modules with `python -m folder.file`, not `python folder/file.py`. This keeps imports like `from database.connection import ...` working everywhere.
 3. **Do not move to the next phase until the current checkpoint passes.**
 4. Commit to Git at the end of every phase.
-

Part 0 — Understanding the System

0.1 What we are building

A user asks:

"Which product category had the largest revenue increase from Q2 to Q3 2017?"

The system:

1. **Understands** the question and rewrites follow-ups ("what about Q4?") into standalone questions.
2. **Asks for clarification** if the question is genuinely ambiguous.
3. **Retrieves business context** with RAG: what "revenue" means, data-quality notes, similar solved examples.
4. **Asks the Knowledge Graph** which tables are involved, how they join, and how metrics are defined.

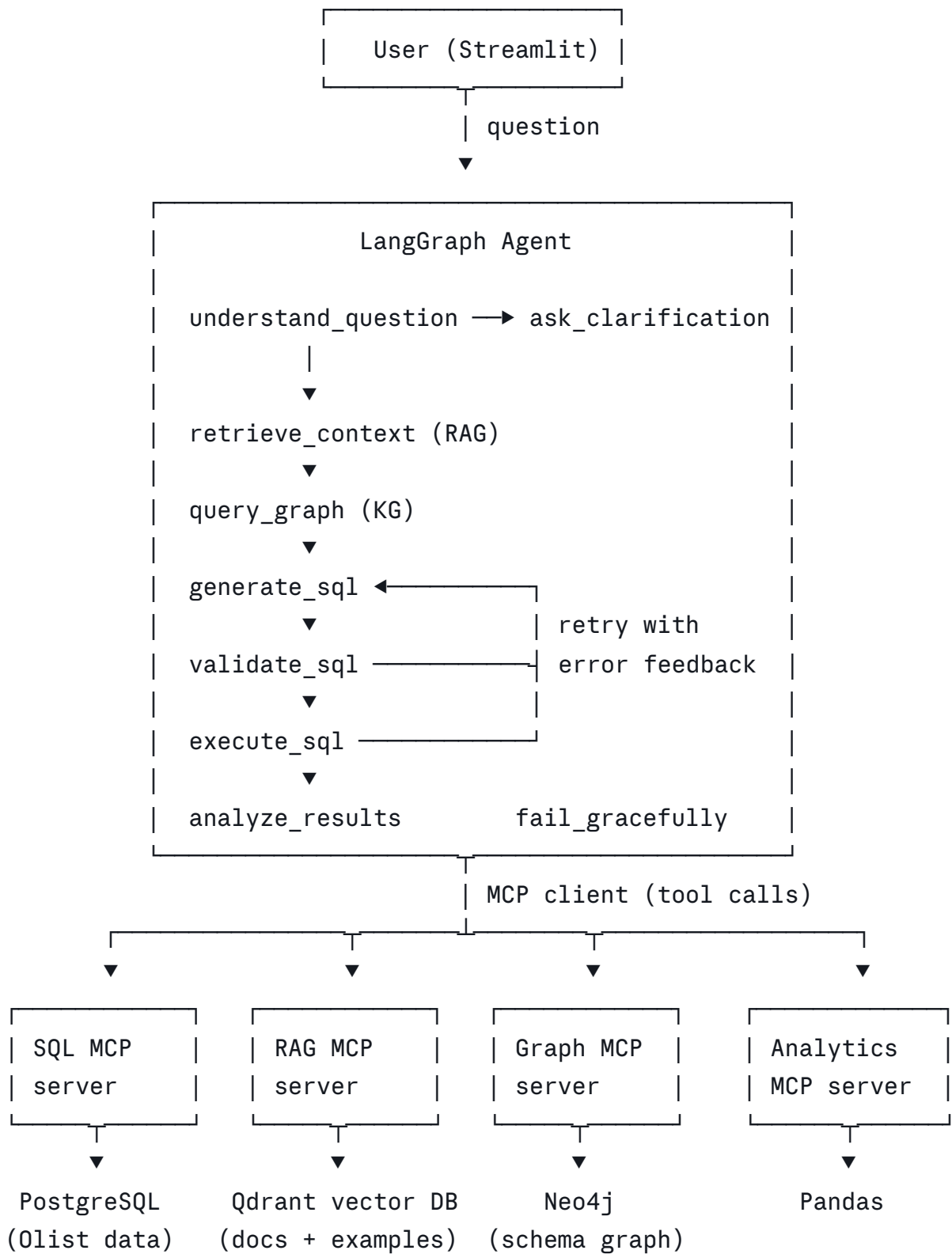
5. **Generates SQL** with an LLM using all that context.
6. **Validates** the SQL: read-only, allowed tables only, row limit.
7. **Executes** it through an **MCP** tool against PostgreSQL.
8. **Self-corrects** if it fails: it reads the error, fixes the SQL and retries (max 3).
9. **Analyzes** the result and writes an answer grounded in the actual numbers.
10. **Shows** the answer, SQL, data table, a chart and the agent's trace in a Streamlit UI.
11. **Measures** everything against a baseline so you can prove each component helped.

0.2 The one-sentence design principle

PostgreSQL stores the facts. RAG retrieves the meaning. The Knowledge Graph stores the relationships. MCP exposes everything as tools. LangGraph decides what happens next.

0.3 Architecture

text



0.4 What each technology does

Technology	Role in this project	Question it answers
PostgreSQL	Stores the Olist data	What are the actual numbers?
SQL	Query language	How do we fetch the numbers?
RAG (embeddings + retrieval)	Retrieves docs, business rules and example queries	What does this data <i>mean</i> ? How were similar questions solved?
Vector database (Qdrant)	Stores the embeddings and runs similarity search	Which stored text is closest in meaning to this question?
Knowledge Graph (Neo4j)	Stores tables, columns, joins, metrics and concepts	How are things connected? Which tables and joins does this question need?
MCP	Standard tool interface between the agent and every capability	How does the agent <i>use</i> the database, RAG and KG safely?
LangGraph	Stateful workflow with branches, retries and memory	What should the agent do next?
LangChain	LLM wrappers, retrievers, text splitters	How do the LLM pieces connect?
Pandas + Plotly	Result analysis and charts	What patterns are in the result?
Streamlit	UI	How does a user interact with it?
LangSmith	Tracing	What exactly did the agent do?
pytest	Tests	Does it still work after changes?
Docker	Runs Postgres, Neo4j and Qdrant	Can someone else run it?

0.5 Why not put everything in the vector database?

A question like "What was total revenue in 2017?" needs an **exact sum over ~100,000 rows**. Similarity search cannot add numbers, so the answer must come from SQL.

The vector database is for **meaning**:

text

```
"Revenue = SUM(order_items.price) for delivered orders, excluding freight
"customer_id is per-order; use customer_unique_id to count real people."
```

The Knowledge Graph is for **structure**:

text

```
customers —JOINS_TO→ orders —JOINS_TO→ order_items —JOINS_TO→ pro
```

0.6 Why Olist?

The Olist dataset is real Brazilian e-commerce data (~100k orders, 2016–2018) in 9 tables. It is messy in ways that make every component *necessary* rather than decorative:

- There are **two ways to measure money**: `order_items.price` and `order_payments.payment_value`. An LLM will pick the wrong one without business rules, which is a job for **RAG**.
- `customer_id` looks like a person ID but is actually per order. That's a trap only **documentation** can fix.
- Reaching "revenue by customer state" needs a **4-table join path**, which is a job for the **Knowledge Graph**.
- Category names are in Portuguese and live in a separate translation table. That's a **value lookup** problem.
- The data has typos, duplicates and missing values, so **self-correction and validation** get exercised for real.

Part 1 — Final Repository Structure

This is where you'll end up. **Don't create it all now**. Each phase tells you exactly which files to create.

text

```
olist-analytics-copilot/
|
├─ app/
|   └─ main.py                # Streamlit UI
|
├─ config/
|   ├── __init__.py
|   └─ settings.py           # All configuration in one place
|
├─ data/
|   └─ raw/                  # Olist CSVs (git-ignored)
```

```

└─ knowledge/                                # Markdown docs for RAG + examples.js
|
└─ database/
|   └─ __init__.py
|   └─ schema.sql                          # Table definitions
|   └─ readonly_role.sql                   # Read-only DB user for the agent
|   └─ connection.py                      # SQLAlchemy engines
|   └─ load_data.py                       # CSV → Postgres loader
|   └─ introspect.py                     # Reads schema + foreign keys from Pos
|   └─ executor.py                       # Runs SELECT queries safely → DataFra
|   └─ sql_validator.py                   # sqlglot-based safety checks
|
└─ llm/
|   └─ __init__.py
|   └─ client.py                          # LLM factory
|   └─ prompts.py                        # Prompt templates
|   └─ sql_generator.py                   # question + context → SQL
|
└─ rag/
|   └─ __init__.py
|   └─ build_value_docs.py                # Auto-generates docs of real column
|   └─ ingest.py                         # Builds the Qdrant collections
|   └─ retriever.py                      # Search functions
|
└─ graph/
|   └─ __init__.py
|   └─ catalog.py                        # Hand-written semantic layer (metrics
|   └─ build_graph.py                    # Builds the Neo4j graph from Postgres
|   └─ queries.py                        # Join paths, metric lookup, context
|
└─ mcp_servers/
|   └─ __init__.py
|   └─ sql_server.py
|   └─ rag_server.py
|   └─ graph_server.py
|   └─ analytics_server.py
|
└─ analytics/
|   └─ __init__.py
|   └─ analysis.py                       # DataFrame summaries
|   └─ visualization.py                  # Chart selection
|
└─ agent/
|   └─ __init__.py
|   └─ state.py                          # LangGraph state definition
|   └─ tools.py                          # MCP client wrapper

```

```

|   ├── nodes.py                # Each step of the workflow
|   ├── edges.py                # Routing decisions
|   └── graph.py                 # Wires nodes + edges together
|
|── scripts/
|   ├── __init__.py
|   ├── ask_baseline.py         # CLI: baseline text-to-SQL
|   ├── test_mcp_client.py      # CLI: call MCP tools by hand
|   └── run_agent.py            # CLI: chat with the full agent
|
|── evaluation/
|   ├── __init__.py
|   ├── questions.json          # Test questions + gold SQL
|   ├── compare.py              # Result comparison logic
|   ├── evaluate_sql.py         # Experiments A-C
|   ├── evaluate_agent.py       # Experiment D (full agent)
|   └── results/                # Saved results (commit these!)
|
|── tests/
|   ├── test_sql_validator.py
|   ├── test_edges.py
|   └── test_compare.py
|
|── finetuning/                  # OPTIONAL Phase 11 (LoRA)
|
|── notebooks/
|   └── 01_explore_olist.ipynb
|
|── .env
|── .gitignore
|── docker-compose.yml
|── requirements.txt
└── README.md

```



Why so many small files? Each file has one job. When something breaks (say, the agent picked the wrong join), you know exactly which file to open: `graph/queries.py`. In interviews, this structure also shows you think in systems rather than in notebooks.

Phase 0 — Environment Setup

Goal: Python environment, Postgres and Neo4j running, configuration in place.

0.1 Create the project and virtual environment

```
bash
```

```
mkdir olist-analytics-copilot
cd olist-analytics-copilot
git init

python3 -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
```

Create every package folder, each with an empty `__init__.py`:

```
bash

mkdir -p app config data/raw data/knowledge database llm rag graph \
        mcp_servers analytics agent scripts evaluation/results tests notes

for d in config database llm rag graph mcp_servers analytics agent scripts
do
    touch $d/__init__.py
done
```



What's happening: An `__init__.py` file marks a folder as a Python *package*. That's what lets `from database.connection import get_engine` work from any other folder.

0.2 `requirements.txt`



File: `requirements.txt`

text

```
# Config
python-dotenv

# Data + database
pandas
sqlalchemy
psycopg2-binary
sqlglot

# LLM + RAG
openai
langchain
langchain-core
langchain-openai
langchain-community
langchain-text-splitters

# Vector database
qdrant-client
langchain-qdrant

# Knowledge graph
neo4j

# MCP + agent
mcp
langchain-mcp-adapters
langgraph
langsmith

# UI + charts
streamlit
plotly

# Dev
jupyter
kaggle
pytest
```

```
bash
```

```
pip install -r requirements.txt
```



Why no LoRA libraries yet? `torch`, `transformers` and `peft` are large

downloads. We add them only if you do the optional LoRA phase.

0.3 Run Postgres and Neo4j with Docker

 **File:** `docker-compose.yml`

yaml

```
services:
  postgres:
    image: postgres:16
    container_name: olist-postgres
    environment:
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: admin
      POSTGRES_DB: olist
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data

  neo4j:
    image: neo4j:5
    container_name: olist-neo4j
    environment:
      NEO4J_AUTH: neo4j/olistgraph123
    ports:
      - "7474:7474"    # browser UI
      - "7687:7687"    # bolt protocol (what Python uses)
    volumes:
      - neo4jdata:/data

  qdrant:
    image: qdrant/qdrant:latest
    container_name: olist-qdrant
    ports:
      - "6333:6333"    # REST API + web dashboard
      - "6334:6334"    # gRPC
    volumes:
      - qdrantdata:/qdrant/storage

volumes:
  pgdata:
  neo4jdata:
  qdrantdata:
```

```
bash
```

```
docker compose up -d
docker compose ps          # all three should show "running"
```

 **What's happening:** Docker runs three databases in isolated containers. Each has a different job:

- **Postgres** stores the facts.
- **Neo4j** stores the relationships. Port `7474` gives you a web UI at <http://localhost:7474>, where you'll see your knowledge graph.
- **Qdrant** stores the embeddings. Its dashboard at <http://localhost:6333/dashboard> lets you browse stored vectors and run searches.

`volumes` keep each database's data on disk, so it survives restarts.

0.4 `.env`

 **File:** `.env`

```
text
```

```
# Admin user: used ONLY for loading data
DATABASE_URL=postgresql+psycopg2://admin:admin@localhost:5432/olist

# Read-only user: used by the agent for every query
READONLY_DATABASE_URL=postgresql+psycopg2://copilot_ro:readonly@localhost

NEO4J_URI=bolt://localhost:7687
NEO4J_USERNAME=neo4j
NEO4J_PASSWORD=olistgraph123

QDRANT_URL=http://localhost:6333

OPENAI_API_KEY=sk-your-key-here
OPENAI_MODEL=gpt-4.1-mini
EMBEDDING_MODEL=text-embedding-3-small
```

 **Why two database users?** This is the most important security decision in the project. The agent's SQL comes from an LLM, and LLMs can be tricked (prompt injection) or simply make mistakes. If the agent connects as a user who *physically cannot* write, then no bug in your validator can ever delete data. The validator is a second layer, not the only one. This is called **defense in depth**.

0.5 `.gitignore`

 **File:** `.gitignore`

text

```
.venv/  
__pycache__/  
.env  
data/raw/  
.ipynb_checkpoints/  
finetuning/output/
```

0.6 Central configuration

 **File:** `config/settings.py`

python

```
"""  
All configuration lives here. Every other module imports from this file  
instead of reading environment variables itself.  
"""  
  
import os  
from pathlib import Path  
  
from dotenv import load_dotenv  
  
# Project root = the folder that contains config/  
ROOT = Path(__file__).resolve().parent.parent  
load_dotenv(ROOT / ".env")  
  
# --- Databases ---  
DATABASE_URL = os.environ["DATABASE_URL"]  
READONLY_DATABASE_URL = os.environ["READONLY_DATABASE_URL"]  
  
NEO4J_URI = os.getenv("NEO4J_URI", "bolt://localhost:7687")  
NEO4J_USERNAME = os.getenv("NEO4J_USERNAME", "neo4j")  
NEO4J_PASSWORD = os.environ["NEO4J_PASSWORD"]  
  
# --- Vector database ---  
QDRANT_URL = os.getenv("QDRANT_URL", "http://localhost:6333")  
DOCS_COLLECTION = "olist_docs"  
EXAMPLES_COLLECTION = "olist_examples"  
  
# --- LLM ---  
OPENAI_MODEL = os.getenv("OPENAI_MODEL", "gpt-4.1-mini")
```

```

EMBEDDING_MODEL = os.getenv("EMBEDDING_MODEL", "text-embedding-3-small")

# --- Paths ---
RAW_DIR = ROOT / "data" / "raw"
KNOWLEDGE_DIR = ROOT / "data" / "knowledge"

# --- Safety / behaviour ---
MAX_ROWS = 1000          # row limit injected into every query
MAX_RETRIES = 3          # self-correction attempts

ALLOWED_TABLES = {
    "customers",
    "sellers",
    "products",
    "category_translation",
    "orders",
    "order_items",
    "order_payments",
    "order_reviews",
    "geolocation",
}

```



What's happening: `load_dotenv` reads `.env` into environment variables.

We compute `ROOT` from this file's location, so paths work no matter which folder you run from.



Why centralize? When you later change the model or the row limit, you change one line. `os.environ["X"]` (not `.getenv`) is used for required values so the program crashes *immediately* with a clear error if one is missing, instead of failing mysteriously later.



Phase 0 checkpoint

```
bash
```

```
python -c "from config import settings; print(settings.ROOT, settings.OPENAI_API_KEY)"
```

Then open <http://localhost:7474> and log in with `neo4j` / `olistgraph123`, and open <http://localhost:6333/dashboard> to confirm Qdrant is up (it will show no collections yet).

```
bash
```

```
git add . && git commit -m "Phase 0: environment, docker, config"
```

Phase 1 — The Data Layer (Olist in PostgreSQL)

Goal: All 9 Olist tables loaded into Postgres with proper keys, a read-only user, and a written list of data quirks.

1.1 Download the dataset

1. Create a Kaggle account → **Settings** → **API** → **Create New Token**. This downloads `kaggle.json`.
2. Move it to `~/.kaggle/kaggle.json` (Windows: `C:\Users\<you>\.kaggle\kaggle.json`)).

bash

```
kaggle datasets download -d olistbr/brazilian-ecommerce -p data/raw --unzip
ls data/raw
```

You should see 9 CSVs:

text

```
olist_customers_dataset.csv      olist_order_reviews_dataset.csv
olist_geolocation_dataset.csv    olist_orders_dataset.csv
olist_order_items_dataset.csv    olist_products_dataset.csv
olist_order_payments_dataset.csv olist_sellers_dataset.csv
product_category_name_translation.csv
```

1.2 Understand the data model

text

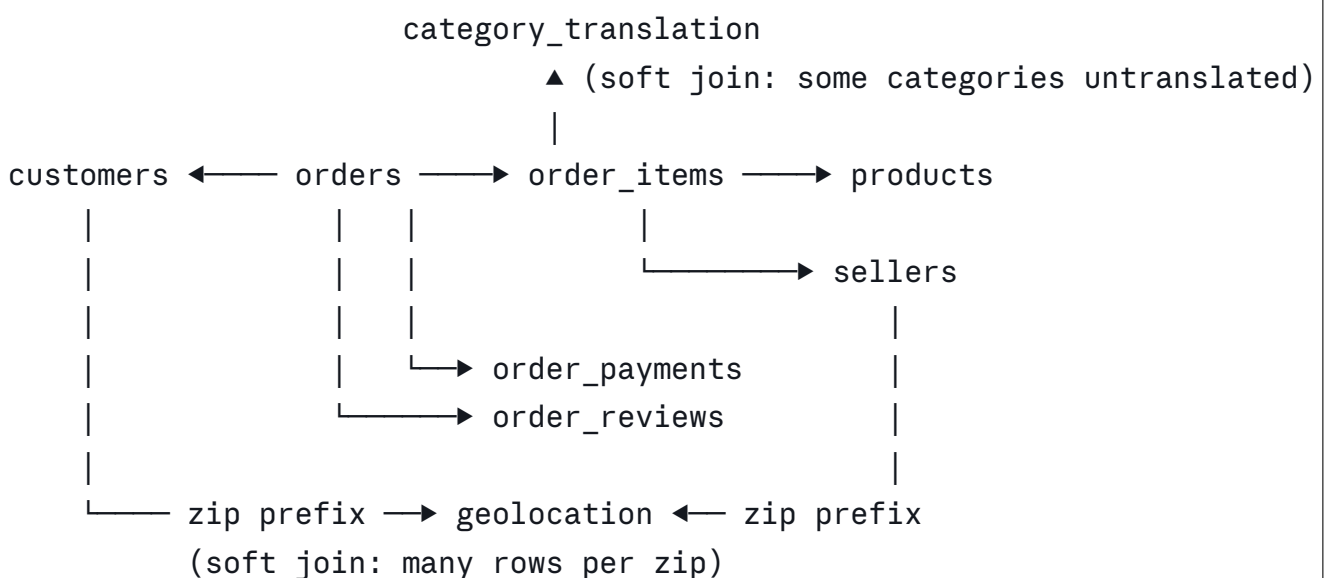


Table	One row =	Key facts
<code>orders</code>	one order	status, purchase and delivery timestamps
<code>order_items</code>	one item line in an order	<code>price</code> , <code>freight_value</code> , product, seller
<code>order_payments</code>	one payment for an order	an order can have several payments
<code>order_reviews</code>	one review	score 1-5, optional comment
<code>customers</code>	one <i>customer-per-order</i> record	<code>customer_unique_id</code> is the real person
<code>products</code>	one product	Portuguese category name
<code>sellers</code>	one seller	location
<code>category_translation</code>	one category	Portuguese → English
<code>geolocation</code>	one lat/lng sample	many rows per zip prefix

1.3 Explore the data in a notebook (do not skip this)

 **File:** `notebooks/01_explore_olist.ipynb`

```
bash
```

```
jupyter notebook notebooks/
```

Run these cells one by one and **read the output**:

```
python
```

```
# Cell 1: load everything
import pandas as pd
RAW = "../data/raw"
customers = pd.read_csv(f"{RAW}/olist_customers_dataset.csv")
orders = pd.read_csv(f"{RAW}/olist_orders_dataset.csv")
items = pd.read_csv(f"{RAW}/olist_order_items_dataset.csv")
payments = pd.read_csv(f"{RAW}/olist_order_payments_dataset.csv")
reviews = pd.read_csv(f"{RAW}/olist_order_reviews_dataset.csv")
products = pd.read_csv(f"{RAW}/olist_products_dataset.csv")
sellers = pd.read_csv(f"{RAW}/olist_sellers_dataset.csv")
geo = pd.read_csv(f"{RAW}/olist_geolocation_dataset.csv")
cats = pd.read_csv(f"{RAW}/product_category_name_translation.csv")

for name, df in [("customers", customers), ("orders", orders), ("items", items), ("payments", payments), ("reviews", reviews), ("products", products), ("sellers", sellers), ("geo", geo), ("cats", cats)]:
    print(f"{name:10s} {df.shape}")
```

python

```
# Cell 2: customer_id vs customer_unique_id
print(customers["customer_id"].nunique(), customers["customer_unique_id"].nunique())
```

python

```
# Cell 3: duplicate review ids?
print(reviews["review_id"].duplicated().sum())
print(reviews.duplicated(subset=["review_id", "order_id"]).sum())
```

python

```
# Cell 4: typo'd column names
print(products.columns.tolist())
```

python

```
# Cell 5: categories with no English translation
missing = set(products["product_category_name"].dropna()) - set(cats["product_category_name_english"].dropna())
print(missing)
```

python

```
# Cell 6: order statuses and date range
print(orders["order_status"].value_counts())
print(orders["order_purchase_timestamp"].min(), orders["order_purchase_ti
```

python

```
# Cell 7: do items and payments agree on money?
item_total = items.groupby("order_id")[["price", "freight_value"]].sum().s
pay_total = payments.groupby("order_id")["payment_value"].sum()
diff = (item_total - pay_total).abs()
print((diff > 1).sum(), "orders where items and payments differ by > 1 BR
```

python

```
# Cell 8: geolocation rows per zip
print(geo.groupby("geolocation_zip_code_prefix").size().describe())
```

python

```
# Cell 9: zip codes lose leading zeros if read as numbers
print(customers["customer_zip_code_prefix"].head())
```

Write down what you find. You should discover roughly:

Quirk	Consequence
~99k <code>customer_id</code> but ~96k <code>customer_unique_id</code>	Counting "customers" with <code>customer_id</code> over-counts. Use <code>customer_unique_id</code> .
Duplicate <code>review_id</code> s	The primary key must be <code>(review_id, order_id)</code> , and exact duplicates are dropped.
<code>product_name_lenght</code> , <code>product_description_lenght</code>	Typos, which we rename on load.
A few categories are missing from the translation table	Use <code>LEFT JOIN</code> + <code>COALESCE</code> , with no foreign key.
Statuses: delivered, shipped, canceled, unavailable, invoiced, processing, created, approved	Revenue should use only <code>delivered</code> .
Data from Sep 2016 to Oct 2018; the edges are sparse	Growth comparisons at the edges are misleading.
Items vs payments totals differ for some orders (vouchers, installments)	Pick ONE revenue definition and document it.
Many geolocation rows per zip	Must aggregate before joining, otherwise rows multiply.
Zip codes read as numbers lose leading zeros	Store them as <code>TEXT</code> .

 **Why this matters:** This table becomes `data/knowledge/data_quality.md`, your RAG content, in Phase 3. It's also excellent interview material: *"I found that `customer_id` was per-order, so the LLM was over-counting customers until I documented it and retrieval fixed it."* That's a real engineering story.

1.4 Table definitions

 **File:** `database/schema.sql`

```
sql

-- Olist e-commerce schema
-- Parent tables first, then child tables (foreign keys need parents to exist)

CREATE TABLE category_translation (
    product_category_name          TEXT PRIMARY KEY,
```

```

        product_category_name_english TEXT
    );

CREATE TABLE customers (
    customer_id TEXT PRIMARY KEY,
    customer_unique_id TEXT NOT NULL,
    customer_zip_code_prefix TEXT,
    customer_city TEXT,
    customer_state CHAR(2)
);

CREATE TABLE sellers (
    seller_id TEXT PRIMARY KEY,
    seller_zip_code_prefix TEXT,
    seller_city TEXT,
    seller_state CHAR(2)
);

-- No foreign key to category_translation, because some categories have no translation

CREATE TABLE products (
    product_id TEXT PRIMARY KEY,
    product_category_name TEXT,
    product_name_length INT,
    product_description_length INT,
    product_photos_qty INT,
    product_weight_g INT,
    product_length_cm INT,
    product_height_cm INT,
    product_width_cm INT
);

CREATE TABLE orders (
    order_id TEXT PRIMARY KEY,
    customer_id TEXT REFERENCES customers(customer_id),
    order_status TEXT,
    order_purchase_timestamp TIMESTAMP,
    order_approved_at TIMESTAMP,
    order_delivered_carrier_date TIMESTAMP,
    order_delivered_customer_date TIMESTAMP,
    order_estimated_delivery_date TIMESTAMP
);

CREATE TABLE order_items (
    order_id TEXT REFERENCES orders(order_id),
    order_item_id INT,
    product_id TEXT REFERENCES products(product_id),

```

```

seller_id          TEXT REFERENCES sellers(seller_id),
shipping_limit_date TIMESTAMP,
price              NUMERIC(12,2),
freight_value      NUMERIC(12,2),
PRIMARY KEY (order_id, order_item_id)
);

CREATE TABLE order_payments (
  order_id          TEXT REFERENCES orders(order_id),
  payment_sequential INT,
  payment_type      TEXT,
  payment_installments INT,
  payment_value     NUMERIC(12,2),
  PRIMARY KEY (order_id, payment_sequential)
);

CREATE TABLE order_reviews (
  review_id         TEXT,
  order_id          TEXT REFERENCES orders(order_id),
  review_score      INT,
  review_comment_title TEXT,
  review_comment_message TEXT,
  review_creation_date TIMESTAMP,
  review_answer_timestamp TIMESTAMP,
  PRIMARY KEY (review_id, order_id)
);

-- Many rows per zip prefix, so this gets a surrogate key and no foreign key
CREATE TABLE geolocation (
  id                SERIAL PRIMARY KEY,
  geolocation_zip_code_prefix TEXT,
  geolocation_lat   NUMERIC(10,6),
  geolocation_lng   NUMERIC(10,6),
  geolocation_city  TEXT,
  geolocation_state CHAR(2)
);

-- Indexes on columns we join and filter on often
CREATE INDEX idx_orders_customer ON orders(customer_id);
CREATE INDEX idx_orders_purchase ON orders(order_purchase_timestamp);
CREATE INDEX idx_items_product ON order_items(product_id);
CREATE INDEX idx_items_seller ON order_items(seller_id);
CREATE INDEX idx_geo_zip ON geolocation(geolocation_zip_code_prefix);

```



What's happening:

- **Primary keys** uniquely identify a row. Some are *composite*: `(order_id, order_item_id)` means "the 2nd item of order X".
- **REFERENCES** creates a **foreign key**, a promise that every `orders.customer_id` exists in `customers`. Postgres enforces it.
- **Indexes** make joins and date filters fast.

 **Why the foreign keys matter so much here:** In Phase 4, we will *automatically read these foreign keys* to build the Knowledge Graph. Good schema design directly feeds your AI system. Where a real relationship exists but can't be a foreign key (geolocation, translation), we'll add it to the graph by hand as a "soft join".

1.5 Read-only role

 **File:** `database/readonly_role.sql`

```
sql

-- Create the role only if it doesn't exist yet
DO $$
BEGIN
    IF NOT EXISTS (SELECT FROM pg_roles WHERE rolname = 'copilot_ro') THEN
        CREATE ROLE copilot_ro LOGIN PASSWORD 'readonly';
    END IF;
END
$$;

GRANT CONNECT ON DATABASE olist TO copilot_ro;
GRANT USAGE ON SCHEMA public TO copilot_ro;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO copilot_ro;

-- Any single query from this user is killed after 15 seconds
ALTER ROLE copilot_ro SET statement_timeout = '15s';
```

 **What's happening:** `copilot_ro` can only `SELECT`. Any `INSERT`, `DROP` or `DELETE` is rejected by Postgres itself. `statement_timeout` protects against runaway queries, such as an accidental cross join of two 100k-row tables.

1.6 Database connections

 **File:** `database/connection.py`

```
python
```

```

"""
Two engines:
- admin (readonly=False): used ONLY by load_data.py
- readonly (default):      used by everything the agent touches
"""

from functools import lru_cache

from sqlalchemy import create_engine, text

from config import settings

@lru_cache
def get_engine(readonly: bool = True):
    url = settings.READONLY_DATABASE_URL if readonly else settings.DATABASE_URL
    return create_engine(url, pool_pre_ping=True)

if __name__ == "__main__":
    with get_engine(readonly=False).connect() as conn:
        print(conn.execute(text("SELECT version()")).scalar())

```

 **What's happening:** An *engine* manages a pool of database connections.

`@lru_cache` means calling `get_engine()` 100 times returns the same engine instead of creating 100 pools. `pool_pre_ping` checks that a connection is alive before using it, which avoids errors after Docker restarts.

 **Why `readonly=True` is the default:** If a developer (you, in 3 weeks) forgets the argument, they get the *safe* engine.

1.7 The loader

 **File:** `database/load_data.py`

```

python

"""
Loads the 9 Olist CSVs into Postgres.

Run:  python -m database.load_data
Re-running is safe: it drops and recreates everything.
"""

import pandas as pd

from config import settings
from database.connection import get_engine

```

```

# (table name, csv file). Order matters: parents before children.
FILES = [
    ("category_translation", "product_category_name_translation.csv"),
    ("customers", "olist_customers_dataset.csv"),
    ("sellers", "olist_sellers_dataset.csv"),
    ("products", "olist_products_dataset.csv"),
    ("orders", "olist_orders_dataset.csv"),
    ("order_items", "olist_order_items_dataset.csv"),
    ("order_payments", "olist_order_payments_dataset.csv"),
    ("order_reviews", "olist_order_reviews_dataset.csv"),
    ("geolocation", "olist_geolocation_dataset.csv"),
]

# Read zip codes as text so leading zeros survive
ZIP_COLUMNS = {
    "customer_zip_code_prefix": str,
    "seller_zip_code_prefix": str,
    "geolocation_zip_code_prefix": str,
}

DATE_SUFFIXES = ("_date", "_timestamp", "_at")

def clean(table: str, df: pd.DataFrame) -> pd.DataFrame:
    """Table-specific fixes discovered in the exploration notebook."""
    if table == "products":
        df = df.rename(columns={
            "product_name_lenght": "product_name_length",
            "product_description_lenght": "product_description_length",
        })

    if table == "order_reviews":
        df = df.drop_duplicates(subset=["review_id", "order_id"])

    # Convert every date-like column to real datetimes
    for col in df.columns:
        if col.endswith(DATE_SUFFIXES):
            df[col] = pd.to_datetime(df[col], errors="coerce")

    return df

def run_sql_file(conn, path):
    conn.exec_driver_sql(path.read_text())

```

```

def load():
    engine = get_engine(readonly=False)
    db_dir = settings.ROOT / "database"

    # 1. Reset the schema and create tables
    with engine.begin() as conn:
        conn.exec_driver_sql("DROP SCHEMA public CASCADE; CREATE SCHEMA public")
        run_sql_file(conn, db_dir / "schema.sql")

    # 2. Load each CSV
    for table, filename in FILES:
        df = pd.read_csv(settings.RAW_DIR / filename, dtype=ZIP_COLUMNS)
        df = clean(table, df)
        df.to_sql(
            table,
            engine,
            if_exists="append",  # tables already exist; just insert rows
            index=False,
            chunksize=5_000,
            method="multi",
        )
        print(f"  loaded {table:22s} {len(df):>9,} rows")

    # 3. (Re)grant read-only access; dropping the schema removed old grants
    with engine.begin() as conn:
        run_sql_file(conn, db_dir / "readonly_role.sql")
        conn.exec_driver_sql("ANALYZE;")

    print("Done.")

if __name__ == "__main__":
    load()

```

What's happening, step by step:

1. `DROP SCHEMA public CASCADE` deletes all tables, so re-running gives a clean slate. `engine.begin()` wraps the block in a transaction that commits automatically.
2. For each CSV: read it with pandas, apply fixes, and insert with `to_sql`. `if_exists="append"` is crucial. It inserts into *our* table (with our keys and types) instead of letting pandas invent its own table.
3. Dropping the schema also dropped the permissions, so we re-grant them. `ANALYZE` updates Postgres's statistics so its query planner makes good choices.



Why `exec_driver_sql` instead of `text()`? SQLAlchemy's `text()` treats `:word` as a parameter placeholder, which would break on SQL containing `::` casts or time literals like `'10:00'`. `exec_driver_sql` sends the SQL as-is.

Run it (geolocation has ~1M rows, so expect 1-3 minutes):

```
bash
```

```
python -m database.load_data
```

1.8 Safe query execution helper

We'll run SELECT queries from several places: the CLI, MCP and evaluation. So we write the logic once.



File: `database/executor.py`

```
python
```

```

"""
Runs a SELECT query as the read-only user and returns a pandas DataFrame.
"""

from decimal import Decimal

import pandas as pd

from database.connection import get_engine

def _decimals_to_float(df: pd.DataFrame) -> pd.DataFrame:
    """Postgres NUMERIC arrives as Python Decimal; convert for pandas/JSON"""
    for col in df.columns:
        if df[col].map(lambda v: isinstance(v, Decimal)).any():
            df[col] = pd.to_numeric(df[col], errors="coerce")
    return df

def run_select(sql: str) -> pd.DataFrame:
    """Execute SQL on the read-only connection. Raises on database errors"""
    conn = get_engine(readonly=True).raw_connection()
    try:
        cursor = conn.cursor()
        cursor.execute(sql)
        columns = [col[0] for col in cursor.description]
        rows = cursor.fetchall()
    finally:
        conn.close() # returns connection to the pool; uncommitted work
    return _decimals_to_float(pd.DataFrame(rows, columns=columns))

```

 **What's happening:** We use a *raw* database cursor so the SQL goes to Postgres exactly as the LLM wrote it. No library tries to interpret `%` in `LIKE '%bed%'` or `:` in timestamps. Postgres `NUMERIC` becomes Python `Decimal`, which pandas and JSON handle poorly, so we convert those columns to floats.

✅ Phase 1 checkpoint

Connect to Postgres and run these queries **by hand**:

```
bash
```

```
docker exec -it olist-postgres psql -U admin -d olist
```

```
sql
```

```
-- 1. Row counts
SELECT 'orders' t, COUNT(*) FROM orders
UNION ALL SELECT 'order_items', COUNT(*) FROM order_items
UNION ALL SELECT 'customers', COUNT(*) FROM customers;

-- 2. Date range
SELECT MIN(order_purchase_timestamp), MAX(order_purchase_timestamp) FROM o

-- 3. Top categories by revenue (4-table join)
SELECT COALESCE(t.product_category_name_english, p.product_category_name)
       ROUND(SUM(oi.price), 2) AS revenue
FROM order_items oi
JOIN orders o ON o.order_id = oi.order_id
JOIN products p ON p.product_id = oi.product_id
LEFT JOIN category_translation t ON t.product_category_name = p.product_c
WHERE o.order_status = 'delivered'
GROUP BY 1
ORDER BY 2 DESC
LIMIT 10;
```

Then verify that the read-only user really is read-only:

```
bash

docker exec -it olist-postgres psql -U copilot_ro -d olist -c "DELETE FROM
# Expected: ERROR: permission denied for table orders
```

 **Save every query you write by hand.** Queries 2 and 3 are the start of your evaluation set in the next phase.

```
bash

git add . && git commit -m "Phase 1: Olist loaded into Postgres with read-
```

Phase 2 — Baseline Text-to-SQL + Evaluation Set

Goal: The simplest possible system: *question* → *LLM (with schema)* → *SQL* → *result*. Plus a scoring script, so every later phase can prove it improved things.

 **Why build a "dumb" version first?** Without a baseline, you can never say "RAG improved accuracy from 47% to 73%". You could only say "I added RAG." Numbers are what make this a top resume project.

2.1 Reading the schema automatically

Instead of pasting the schema into prompts by hand, we read it from Postgres. That's

what makes the system work on *any* database later.

 **File:** `database/introspect.py`

```
python
```

```
"""
Reads table/column/foreign-key information directly from Postgres.
Used by: the baseline prompt, the SQL MCP server, and the Knowledge Graph
"""

from collections import defaultdict

from sqlalchemy import text

from database.connection import get_engine

COLUMNS_SQL = """
SELECT table_name, column_name, data_type
FROM information_schema.columns
WHERE table_schema = 'public'
ORDER BY table_name, ordinal_position
"""

# pg_catalog works for any user, unlike information_schema.table_constraints
# which hides constraints from users who only have SELECT
FOREIGN_KEYS_SQL = """
SELECT
    c.conrelid::regclass::text AS table_name,
    a.attname AS column_name,
    c.confrelid::regclass::text AS foreign_table,
    af.attname AS foreign_column
FROM pg_constraint c
JOIN pg_attribute a ON a.attrelid = c.conrelid AND a.attnum = ANY(c.conkey)
JOIN pg_attribute af ON af.attrelid = c.confrelid AND af.attnum = ANY(c.confkey)
WHERE c.contype = 'f'
    AND c.connamespace = 'public'::regnamespace
"""

def get_columns() -> dict[str, list[tuple[str, str]]]:
    """{table: [(column, type), ...]}"""
    result = defaultdict(list)
    with get_engine().connect() as conn:
        for table, column, dtype in conn.execute(text(COLUMNS_SQL)):
            result[table].append((column, dtype))
    return dict(result)
```

```

def get_foreign_keys() -> list[dict]:
    with get_engine().connect() as conn:
        rows = conn.execute(text(FOREIGN_KEYS_SQL)).mappings().all()
    return [dict(r) for r in rows]

def get_schema_text(tables: list[str] | None = None) -> str:
    """Compact, LLM-friendly schema description."""
    columns = get_columns()
    fks = get_foreign_keys()
    wanted = set(tables) if tables else set(columns)

    lines = []
    for table in sorted(wanted & set(columns)):
        cols = ", ".join(f"{c} {t}" for c, t in columns[table])
        lines.append(f"TABLE {table} ({cols})")

    lines.append("")
    lines.append("FOREIGN KEYS:")
    for fk in fks:
        if fk["table_name"] in wanted or fk["foreign_table"] in wanted:
            lines.append(
                f"    {fk['table_name']}.{fk['column_name']} -> "
                f"{fk['foreign_table']}.{fk['foreign_column']}"
            )
    return "\n".join(lines)

if __name__ == "__main__":
    print(get_schema_text())

```



What's happening: Every Postgres database describes itself.

`information_schema.columns` lists the columns, and `pg_catalog.pg_constraint` lists the foreign keys. We format that into compact text an LLM can read. The `tables` argument lets us send only *some* tables. That becomes important in Phase 4 (**schema pruning**).



A real-world gotcha we avoided: `information_schema.table_constraints` silently returns *nothing* for a user that only has SELECT. If we'd used it, the read-only user would see zero foreign keys and the knowledge graph would be empty, with no error message. `pg_catalog` doesn't have this restriction.

bash

```
python -m database.introspect
```

2.2 The LLM client

 **File:** `llm/client.py`

```
python
```

```
from langchain_openai import ChatOpenAI
```

```
from config import settings
```

```
def get_llm(temperature: float = 0.0) -> ChatOpenAI:  
    return ChatOpenAI(model=settings.OPENAI_MODEL, temperature=temperature)
```



Why `temperature=0`? SQL generation should be as deterministic as possible. The same question should give the same query, which also makes evaluation repeatable.

2.3 Prompts

 **File:** `llm/prompts.py`

```
python
```

```
SQL_SYSTEM_PROMPT = """You are an expert PostgreSQL analyst for the Olist Brazilian e-commerce database. You write a single, correct, read-only PostgreSQL SELECT query that answers the user's question.
```

Rules:

- Output ONLY the SQL query. No explanation, no markdown.
- Use only tables and columns that appear in the schema.
- Prefer explicit JOINS with ON clauses.
- Round money and averages to 2 decimals.
- If documentation or business rules are provided, FOLLOW THEM. They override your assumptions.

```
"""
```

```
def build_sql_prompt(
    question: str,
    schema: str,
    rag_context: str = "",
    examples: str = "",
    graph_context: str = "",
    error_feedback: str = "",
) -> str:
    """Assemble the user prompt. Empty sections are skipped."""
    sections = [f"### Database schema\n{schema}"]

    if graph_context:
        sections.append(f"### Relevant tables, join paths and metric definitions\n{graph_context}")
    if rag_context:
        sections.append(f"### Documentation and business rules\n{rag_context}")
    if examples:
        sections.append(f"### Similar solved examples\n{examples}")
    if error_feedback:
        sections.append(f"### Your previous attempt failed\n{error_feedback}")

    sections.append(f"### Question\n{question}\n\n### SQL")
    return "\n\n".join(sections)
```



What's happening: One function builds the prompt for every experiment.

The baseline passes only `schema`, RAG adds `rag_context` and `examples`, the KG adds `graph_context`, and self-correction adds `error_feedback`.



Why one function? Fair comparison. If each experiment had a different prompt, you couldn't tell whether the improvement came from RAG or from better wording.

2.4 The SQL generator

 **File:** llm/sql_generator.py

```
python

import re

from langchain_core.messages import HumanMessage, SystemMessage

from llm.client import get_llm
from llm.prompts import SQL_SYSTEM_PROMPT, build_sql_prompt


def clean_sql(raw: str) -> str:
    """LLMs sometimes wrap SQL in ```sql fences. Strip them."""
    match = re.search(r"```(?:sql)?\s*(.*?)```", raw, re.DOTALL | re.IGNORECASE)
    sql = match.group(1) if match else raw
    return sql.strip().rstrip(";").strip()


def generate_sql(question: str, schema: str, **context) -> str:
    """context may include rag_context, examples, graph_context, error_feedback"""
    prompt = build_sql_prompt(question, schema, **context)
    response = get_llm().invoke([
        SystemMessage(content=SQL_SYSTEM_PROMPT),
        HumanMessage(content=prompt),
    ])
    return clean_sql(response.content)
```

 **What's happening:** A *system message* sets the LLM's role and rules. The *human message* carries the specific question and context. `clean_sql` is a defensive step, because even when told "no markdown", models sometimes add fences.

2.5 A CLI to try it

 **File:** scripts/ask_baseline.py

```
python
```

```

"""
Usage: python -m scripts.ask_baseline "How many orders were delivered?"
"""

import sys

from database.executor import run_select
from database.introspect import get_schema_text
from llm.sql_generator import generate_sql


def main():
    question = " ".join(sys.argv[1:]) or "How many orders are there?"
    sql = generate_sql(question, get_schema_text())

    print("\nSQL:\n", sql, "\n")
    try:
        print(run_select(sql).head(20).to_string())
    except Exception as exc:
        print("ERROR:", exc)


if __name__ == "__main__":
    main()

```

bash

```

python -m scripts.ask_baseline "How many orders are there?"
python -m scripts.ask_baseline "How many unique customers have placed orders?"
python -m scripts.ask_baseline "What was total revenue in 2017?"

```

 **Look carefully at the output.** For "unique customers", the baseline will probably count `customer_id`, which is the wrong answer. For "revenue", it may include canceled orders, add freight, or use `order_payments`. **These mistakes are the reason RAG and the KG exist.** Note them down.

2.6 The evaluation set

 **File:** `evaluation/questions.json`

```

json

[
  {
    "id": "q01",
    "question": "How many orders are there in total?",

```

```

    "gold_sql": "SELECT COUNT(*) FROM orders",
    "tags": ["simple"]
  },
  {
    "id": "q02",
    "question": "How many unique customers have placed orders?",
    "gold_sql": "SELECT COUNT(DISTINCT c.customer_unique_id) FROM orders o",
    "tags": ["trap", "join"]
  },
  {
    "id": "q03",
    "question": "Which 5 states have the most customers?",
    "gold_sql": "SELECT customer_state, COUNT(DISTINCT customer_unique_id) FROM orders o",
    "tags": ["trap", "group_by"]
  },
  {
    "id": "q04",
    "question": "What was total revenue in 2017?",
    "gold_sql": "SELECT ROUND(SUM(oi.price), 2) FROM order_items oi JOIN orders o ON oi.order_id = o.order_id",
    "tags": ["business_rule", "date"]
  },
  {
    "id": "q05",
    "question": "What are the top 5 product categories by revenue?",
    "gold_sql": "SELECT COALESCE(t.product_category_name_english, p.product_category_name) AS category, SUM(oi.price) AS revenue FROM order_items oi JOIN orders o ON oi.order_id = o.order_id JOIN product_category_translation t ON o.product_category_name = t.product_category_name_english",
    "tags": ["business_rule", "multi_join", "translation"]
  },
  {
    "id": "q06",
    "question": "What is the average review score?",
    "gold_sql": "SELECT ROUND(AVG(review_score), 2) FROM order_reviews",
    "tags": ["simple"]
  },
  {
    "id": "q07",
    "question": "What is the most common payment type?",
    "gold_sql": "SELECT payment_type, COUNT(*) AS n FROM order_payments GROUP BY payment_type",
    "tags": ["group_by"]
  },
  {
    "id": "q08",
    "question": "What is the average delivery time in days for delivered orders?",
    "gold_sql": "SELECT ROUND(AVG(EXTRACT(EPOCH FROM (order_delivered_customer_date - order_purchase_timestamp))), 2) FROM orders",
    "tags": ["date_math"]
  },
  {

```

```

    "id": "q09",
    "question": "Who are the top 5 sellers by revenue?",
    "gold_sql": "SELECT oi.seller_id, ROUND(SUM(oi.price), 2) AS revenue FROM orders oi",
    "tags": ["business_rule"]
  },
  {
    "id": "q10",
    "question": "Show monthly revenue for 2018.",
    "gold_sql": "SELECT DATE_TRUNC('month', o.order_purchase_timestamp) AS month, SUM(oi.price) AS revenue FROM orders o JOIN orders_items oi ON o.order_id = oi.order_id WHERE o.order_purchase_timestamp >= '2018-01-01' AND o.order_purchase_timestamp <= '2018-12-31'",
    "tags": ["time_series", "business_rule"]
  },
  {
    "id": "q11",
    "question": "What percentage of delivered orders arrived later than the expected delivery date?",
    "gold_sql": "SELECT ROUND(100.0 * AVG(CASE WHEN order_delivered_customer_date > order_expected_delivery_date THEN 1 ELSE 0), 2) AS percentage FROM orders",
    "tags": ["case_when"]
  },
  {
    "id": "q12",
    "question": "Compare the average review score of late deliveries versus on-time deliveries.",
    "gold_sql": "SELECT CASE WHEN o.order_delivered_customer_date > o.order_expected_delivery_date THEN 'Late' ELSE 'On-time' END AS delivery_status, AVG(r.review_score) AS avg_score FROM orders o JOIN reviews r ON o.order_id = r.order_id",
    "tags": ["case_when", "join"]
  },
  {
    "id": "q13",
    "question": "Which product category had the largest revenue increase from 2017 to 2018?",
    "gold_sql": "WITH q AS (SELECT COALESCE(t.product_category_name_english, 'Other') AS category, SUM(oi.price) AS revenue FROM orders o JOIN orders_items oi ON o.order_id = oi.order_id WHERE o.order_purchase_timestamp <= '2017-12-31' UNION ALL SELECT COALESCE(t.product_category_name_english, 'Other') AS category, SUM(oi.price) AS revenue FROM orders o JOIN orders_items oi ON o.order_id = oi.order_id WHERE o.order_purchase_timestamp >= '2018-01-01' AND o.order_purchase_timestamp <= '2018-12-31') SELECT category, revenue_2018 - revenue_2017 AS revenue_increase FROM q",
    "tags": ["cte", "growth", "multi_join"]
  },
  {
    "id": "q14",
    "question": "What are the top 5 customer states by revenue?",
    "gold_sql": "SELECT c.customer_state, ROUND(SUM(oi.price), 2) AS revenue FROM customers c JOIN orders o ON c.customer_id = o.customer_id JOIN orders_items oi ON o.order_id = oi.order_id",
    "tags": ["multi_join", "business_rule"]
  },
  {
    "id": "q15",
    "question": "How many customers made more than one order?",
    "gold_sql": "SELECT COUNT(*) FROM (SELECT c.customer_unique_id FROM customers c JOIN orders o ON c.customer_id = o.customer_id GROUP BY c.customer_unique_id HAVING COUNT(o.order_id) > 1)",
    "tags": ["trap", "subquery"]
  }
]

```



How to grow this set: Aim for **30-50 questions** by the end. Tag each one (`trap`, `multi_join`, `business_rule`, ...), so you can later say "RAG fixed 6/7"

trap questions". Run every gold SQL by hand first to make sure it's correct. A wrong gold answer silently ruins your metrics.

2.7 Comparing results

 **File:** `evaluation/compare.py`

```
python

"""
Execution accuracy: two queries are 'equal' if they return the same data,
even if the SQL text is different.
"""

import pandas as pd

def _normalize_value(value):
    if isinstance(value, float):
        return round(value, 2)
    if isinstance(value, pd.Timestamp):
        return value.isoformat()
    return value

def normalize(df: pd.DataFrame) -> list[tuple]:
    """Ignore column names, column order and row order."""
    rows = []
    for row in df.itertuples(index=False):
        values = [_normalize_value(v) for v in row]
        rows.append(tuple(sorted(values, key=str)))
    return sorted(rows, key=str)

def results_match(predicted: pd.DataFrame, gold: pd.DataFrame) -> bool:
    return normalize(predicted) == normalize(gold)
```

 **What's happening:** There are many correct ways to write the same SQL (different aliases, different join order), so comparing SQL *text* is misleading. Instead, we run both queries and compare *results*. This is called **execution accuracy**, the standard metric in text-to-SQL research (Spider, BIRD).

 **Known limitation:** If the prediction returns extra columns (e.g. `category, q2, q3, increase`) instead of `(category, increase)`, it counts as wrong. That's acceptable, but mention it in your README.

 **File:** `tests/test_compare.py`

python

```
import pandas as pd
```

```
from evaluation.compare import results_match
```

```
def test_same_data_different_order():
```

```
    a = pd.DataFrame({"x": ["b", "a"], "y": [2.0, 1.0]})
```

```
    b = pd.DataFrame({"name": ["a", "b"], "val": [1.0, 2.0]})
```

```
    assert results_match(a, b)
```

```
def test_float_rounding():
```

```
    a = pd.DataFrame({"v": [1.004]})
```

```
    b = pd.DataFrame({"v": [1.0]})
```

```
    assert results_match(a, b)
```

```
def test_different_data():
```

```
    assert not results_match(pd.DataFrame({"v": [1]}), pd.DataFrame({"v":
```

2.8 The experiment runner

 **File:** `evaluation/evaluate_sql.py`

python

```
"""
```

```
Experiments A-C: SQL generation with increasing context.
```

```
Usage:
```

```
python -m evaluation.evaluate_sql --mode baseline           # A: schema only
```

```
python -m evaluation.evaluate_sql --mode rag                # B: + RAG
```

```
python -m evaluation.evaluate_sql --mode rag_kg            # C: + KG
```

```
"""
```

```
import argparse
```

```
import json
```

```
import time
```

```
from config import settings
```

```
from database.executor import run_select
```

```
from database.introspect import get_schema_text
```

```
from evaluation.compare import results_match
```

```
from llm.sql_generator import generate_sql
```

```
EVAL_DIR = settings.ROOT / "evaluation"
```

```
def build_context(mode: str, question: str) -> dict:
    """Imports are inside, so the baseline works before RAG/KG exist."""
    context = {}
    if mode in ("rag", "rag_kg"):
        from rag.retriever import format_docs, format_examples, search_docs
        context["rag_context"] = format_docs(search_docs(question))
        context["examples"] = format_examples(search_examples(question))
    if mode == "rag_kg":
        from graph.queries import build_graph_context
        context["graph_context"] = build_graph_context(question)["context"]
    return context

def evaluate(mode: str):
    questions = json.loads((EVAL_DIR / "questions.json").read_text())
    schema = get_schema_text()
    results = []

    for item in questions:
        start = time.time()
        record = {"id": item["id"], "question": item["question"], "tags": item["tags"]}
        try:
            sql = generate_sql(item["question"], schema, **build_context(mode, item["question"]))
            record["sql"] = sql
            predicted = run_select(sql)
            gold = run_select(item["gold_sql"])
            record["correct"] = results_match(predicted, gold)
        except Exception as exc:
            record["correct"] = False
            record["error"] = str(exc).splitlines()[0]
        record["latency_s"] = round(time.time() - start, 2)
        results.append(record)
        print(f"{'✅' if record['correct'] else '❌'} {item['id']} {item['question']}")

    accuracy = sum(r["correct"] for r in results) / len(results)
    print(f"\nMode: {mode} Execution accuracy: {accuracy:.1%} ({len(results)} items)")

    out = EVAL_DIR / "results" / f"{mode}.json"
    out.write_text(json.dumps({"mode": mode, "accuracy": accuracy, "results": results}))
    print(f"Saved → {out}")
```

```
if __name__ == "__main__":
```

```
parser = argparse.ArgumentParser()
parser.add_argument("--mode", choices=["baseline", "rag", "rag_kg"],
evaluate(parser.parse_args().mode)
```

✅ Phase 2 checkpoint (Milestone 1)

bash

```
pytest tests/test_compare.py
python -m evaluation.evaluate_sql --mode baseline
```

Write the baseline accuracy in your README. Open `evaluation/results/baseline.json` and study **which** questions failed and **why**. Those failures are your roadmap for the next two phases.

bash

```
git add . && git commit -m "Phase 2: baseline text-to-SQL + evaluation ha"
```

Phase 3 — RAG (Retrieval-Augmented Generation)

Goal: Give the LLM the *meaning* of the data (business rules, data quirks, valid values) and *similar solved examples*, retrieved by semantic similarity.

3.1 What RAG is, concretely

text

INDEXING (once)

markdown docs

|



split into chunks

|



embed each chunk → Qdrant (vector DB) (text → vector)

RETRIEVAL (every question)

"total revenue in 2017?"

|



embed question

|



nearest-neighbour search

|



top-k chunks → into the prompt

An **embedding** turns text into a list of ~1,500 numbers, so that texts with similar *meaning* end up close together. "How much money did we make" lands near "Revenue definition" even with zero shared words.

A **vector database** stores those vectors and finds the closest ones to a query vector,

fast. We use **Qdrant**, which runs as its own server in Docker.

Term	Qdrant meaning	Relational analogy
Collection	A named set of vectors, all with the same dimension	A table
Point	One stored item: an id + a vector + a payload	A row
Payload	JSON attached to a point (the chunk text and metadata)	The row's other columns
Distance metric	How "close" is measured (we use cosine similarity)	—



Why a vector database and not a library like FAISS? FAISS is a search *library*. It builds an index inside your Python process and you save it as files. Qdrant is a *database server*. It gives you:

- **Persistence and a network API**, so the MCP server, the evaluation scripts and the app all share one store instead of each loading files.
- **Payload filtering**, e.g. "search only chunks where `table = 'orders'`", combined with similarity search in one query.
- **A dashboard**, to inspect what's stored and debug retrieval visually.
- **No pickle files**, so there's no unsafe deserialization step.

The trade-off is one more service to run. For a system meant to look and behave like production, that's worth it.

We build **two** collections:

Collection	Contents	Why
<code>olist_docs</code>	Table docs, metrics, data quality, glossary, real values	Teaches the meaning and rules
<code>olist_examples</code>	Question → SQL pairs	Shows the <i>pattern</i> for similar questions (few-shot prompting)

3.2 Write the knowledge documents

These are the heart of RAG. **Quality here matters more than any code.**



File: `data/knowledge/orders.md`

markdown

Table: orders

One row per customer order. The central table: almost every question joins

Columns

- order_id: unique order identifier (primary key)
- customer_id: links to customers.customer_id (this id is PER ORDER, not per customer)
- order_status: delivered, shipped, canceled, unavailable, invoiced, processing
- order_purchase_timestamp: when the customer placed the order. USE THIS
- order_approved_at: payment approval time
- order_delivered_carrier_date: handed to the logistics partner
- order_delivered_customer_date: actually delivered to the customer (NULL if not delivered)
- order_estimated_delivery_date: delivery date promised at purchase

Joins

- orders.customer_id = customers.customer_id
- order_items.order_id = orders.order_id
- order_payments.order_id = orders.order_id
- order_reviews.order_id = orders.order_id

Rules

- Revenue and sales questions must filter order_status = 'delivered'.
- A delivery is late when order_delivered_customer_date > order_estimated_delivery_date
- Delivery time in days = EXTRACT(EPOCH FROM (order_delivered_customer_date - order_purchase_timestamp)) / 86400

 **File:** data/knowledge/order_items.md

markdown

Table: order_items

One row per item line inside an order. An order with 3 items has 3 rows (order_item_id = 1, 2, 3).

Columns

- order_id, order_item_id: composite primary key
- product_id: links to products.product_id
- seller_id: links to sellers.seller_id
- shipping_limit_date: seller's deadline to hand the item to the carrier
- price: item price in BRL (Brazilian real). THIS IS THE REVENUE COLUMN.
- freight_value: shipping cost for this item in BRL

Rules

- Revenue = SUM(order_items.price), joined to orders with order_status =
- GMV (gross merchandise value) = SUM(price + freight_value).
- Number of items sold = COUNT(*) on order_items, NOT COUNT(orders).
- To get revenue by category, join order_items → products → category_trans
- To get revenue by customer location, join order_items → orders → customer

 **File:** data/knowledge/order_payments.md

markdown

Table: order_payments

One row per payment. An order can be paid in several parts (e.g. voucher so there can be multiple rows per order (payment_sequential = 1, 2, ...)).

Columns

- order_id, payment_sequential: composite primary key
- payment_type: credit_card, boleto, voucher, debit_card, not_defined
- payment_installments: number of installments chosen by the customer
- payment_value: amount paid in this payment, in BRL

Rules

- Do NOT use payment_value for revenue. Use order_items.price (see metrics payment_value includes freight, and totals differ from item totals for s
- Use this table only for questions about payment methods or installments
- Joining order_payments and order_items in the same query duplicates rows. Aggregate one of them in a subquery or CTE first.
- "boleto" is a Brazilian bank-slip payment method.

 **File:** data/knowledge/order_reviews.md

markdown

Table: order_reviews

One row per customer review of an order.

Columns

- review_id, order_id: composite primary key (review_id alone is NOT unique)
- review_score: integer 1 (worst) to 5 (best)
- review_comment_title, review_comment_message: optional free text in Portuguese
- review_creation_date: when the review survey was sent
- review_answer_timestamp: when the customer answered

Rules

- "Customer satisfaction" or "rating" means review_score.
- Average rating = ROUND(AVG(review_score), 2).
- A few orders have more than one review; for per-order analysis use AVG

 **File:** data/knowledge/customers.md

markdown

Table: customers

IMPORTANT: one row per customer PER ORDER, not per person.

Columns

- customer_id: key used by orders.customer_id. A new customer_id is created for each order.
- customer_unique_id: the real, unique person. USE THIS to count customers
- customer_zip_code_prefix: first 5 digits of the zip code (text, keep leading zeros)
- customer_city: city name in lowercase Portuguese, e.g. 'sao paulo'
- customer_state: 2-letter Brazilian state code, e.g. 'SP'

Rules

- "How many customers" → COUNT(DISTINCT customer_unique_id).
- "Repeat customers" → customer_unique_id with COUNT(DISTINCT order_id) > 1
- Customer location questions use customer_state / customer_city (no need for customer_zip_code_prefix)

 **File:** data/knowledge/sellers.md

markdown

Table: sellers

One row per seller (merchant) on the Olist marketplace.

Columns

- seller_id: primary key; linked from order_items.seller_id
- seller_zip_code_prefix, seller_city, seller_state: seller location

Rules

- Seller revenue = SUM(order_items.price) GROUP BY seller_id, delivered on
- Sellers have no name column; answers identify them by seller_id.
- Seller-to-customer distance needs geolocation (see geolocation doc).

 **File:** `data/knowledge/products.md`

markdown

Table: products

One row per product.

Columns

- product_id: primary key
- product_category_name: category in PORTUGUESE (e.g. 'cama_mesa_banho').
- product_name_length, product_description_length: character counts (not
- product_photos_qty: number of listing photos
- product_weight_g, product_length_cm, product_height_cm, product_width_cm

Rules

- To show English category names, LEFT JOIN category_translation and use COALESCE(t.product_category_name_english, p.product_category_name) AS ca
- Products have no name or brand columns.

 **File:** `data/knowledge/category_translation.md`

markdown

Table: category_translation

Maps Portuguese category names to English.

Columns

- product_category_name: Portuguese name (primary key)
- product_category_name_english: English name

Rules

- ALWAYS use LEFT JOIN (not INNER JOIN): a few categories have no translation and an inner join would silently drop their revenue.
- When the user mentions a category in English (e.g. "health beauty", "food"), filter on product_category_name_english.

 **File:** `data/knowledge/geolocation.md`

markdown

Table: geolocation

Latitude/longitude samples per zip code prefix. MANY rows per zip prefix.

Columns

- geolocation_zip_code_prefix, geolocation_lat, geolocation_lng, geolocation_timestamp

Rules

- NEVER join geolocation directly to customers or sellers. Each zip has many rows, so the join multiplies rows and inflates every SUM and COUNT.
- First aggregate: `SELECT geolocation_zip_code_prefix, AVG(geolocation_lat) lat, AVG(geolocation_lng) lng FROM geolocation GROUP BY 1`. Then join on the aggregated table.
- For state- or city-level questions, use customers/sellers columns instead of geolocation.

 **File:** `data/knowledge/metrics.md`

markdown

```
# Business metric definitions
```

```
## Revenue (also: sales, turnover, income)
```

```
SUM(order_items.price) for orders with order_status = 'delivered'.  
Excludes freight. Date = orders.order_purchase_timestamp.
```

```
## GMV (gross merchandise value)
```

```
SUM(order_items.price + order_items.freight_value), delivered orders.
```

```
## Number of orders
```

```
COUNT(DISTINCT orders.order_id).
```

```
## Average order value (AOV)
```

```
Revenue / COUNT(DISTINCT order_id), delivered orders.
```

```
## Number of customers
```

```
COUNT(DISTINCT customers.customer_unique_id).
```

```
## Average review score
```

```
ROUND(AVG(order_reviews.review_score), 2).
```

```
## Late delivery rate
```

```
Share of delivered orders where order_delivered_customer_date > order_esti
```

```
## Quarters
```

```
Calendar quarters: Q1 = Jan-Mar, Q2 = Apr-Jun, Q3 = Jul-Sep, Q4 = Oct-Dec  
Filter with half-open ranges: ts >= '2017-07-01' AND ts < '2017-10-01'.
```

 **File:** `data/knowledge/data_quality.md`

markdown

```
# Data quality notes
```

- Data covers roughly Sep 2016 to Oct 2018. 2016 and the last months of 2018 growth comparisons involving them are misleading.
- customer_id is per order; customer_unique_id is per person.
- review_id is not unique on its own.
- Some product categories are NULL or have no English translation.
- Order item totals and payment totals differ for some orders (vouchers, etc).
- geolocation has many rows per zip prefix and must be aggregated before use.
- All money is in BRL (Brazilian real).
- City names are lowercase Portuguese without accents in most rows (e.g. São Paulo).

 **File:** `data/knowledge/glossary.md`

markdown

Glossary

Brazilian states (most common)

SP = São Paulo, RJ = Rio de Janeiro, MG = Minas Gerais, RS = Rio Grande do Sul, PR = Paraná, SC = Santa Catarina, BA = Bahia, DF = Distrito Federal (Brasília), GO = Goiás, ES = Espírito Santo, PE = Pernambuco, CE = Ceará.

Terms

- Boleto: bank-slip payment method common in Brazil.
- Freight: shipping cost (order_items.freight_value).
- Seller: a merchant selling through the Olist marketplace.
- Installments: number of monthly payments a purchase is split into.



Why each doc has a "Rules" section: Retrieval returns *chunks*. If the revenue rule only lived in `metrics.md`, a question like "top sellers" might retrieve `sellers.md` but not `metrics.md`. Repeating the key rule where it's relevant makes retrieval robust. That's a deliberate, explainable design choice.

3.3 Auto-generate "real values" docs

The LLM can't know that the category is `'beleza_saude'` in Portuguese or that the status is `'delivered'` and not `'Delivered'`. So we pull real values from the database.



File: `rag/build_value_docs.py`

python

"""

Writes data/knowledge/generated_values.md with real values from the database so the LLM uses exact spellings in WHERE clauses.

Run: `python -m rag.build_value_docs`

"""

from config import settings

from database.executor import run_select

def section(title: str, sql: str, fmt) -> str:

df = run_select(sql)

lines = [f"## {title}"] + [fmt(row) for row in df.itertuples(index=False)]

return "\n".join(lines)

```

def build():
    parts = ["# Actual values in the database (auto-generated)\n"]

    parts.append(section(
        "Order status values",
        "SELECT order_status, COUNT(*) n FROM orders GROUP BY 1 ORDER BY 1",
        lambda r: f"- '{r[0]}' ({r[1]:,} orders)",
    ))

    parts.append(section(
        "Payment types",
        "SELECT payment_type, COUNT(*) n FROM order_payments GROUP BY 1 ORDER BY 1",
        lambda r: f"- '{r[0]}' ({r[1]:,} payments)",
    ))

    parts.append(section(
        "Product categories (Portuguese → English)",
        """SELECT DISTINCT p.product_category_name, t.product_category_name_english
        FROM products p
        LEFT JOIN category_translation t USING (product_category_name)
        WHERE p.product_category_name IS NOT NULL
        ORDER BY 1""",
        lambda r: f"- '{r[0]}' → '{r[1] or 'NO TRANSLATION'}'",
    ))

    parts.append(section(
        "Customer states",
        "SELECT customer_state, COUNT(DISTINCT customer_unique_id) n FROM customers",
        lambda r: f"- '{r[0]}' ({r[1]:,} customers)",
    ))

    out = settings.KNOWLEDGE_DIR / "generated_values.md"
    out.write_text("\n\n".join(parts))
    print(f"Wrote {out}")

if __name__ == "__main__":
    build()

```

 **What's happening:** This is **value retrieval**. When someone asks about "health and beauty products", RAG retrieves the line `'beleza_saude' → 'health_beauty'`, and the LLM writes `WHERE product_category_name_english = 'health_beauty'` correctly.

 **Why generate instead of hand-write:** It's always in sync with the data, and it's the first step toward making this work on *any* database.

3.4 Few-shot examples

 **File:** `data/knowledge/examples.json`

json

```
[
  {
    "question": "How many sellers are there in each state?",
    "sql": "SELECT seller_state, COUNT(*) AS sellers FROM sellers GROUP BY seller_state",
  },
  {
    "question": "What is the average freight value per product category?",
    "sql": "SELECT COALESCE(t.product_category_name_english, p.product_category_name) AS product_category, AVG(freight_value) AS avg_freight FROM product_reviews t JOIN product_category_name p ON t.product_category_name_english = p.product_category_name",
  },
  {
    "question": "How many orders are in each status?",
    "sql": "SELECT order_status, COUNT(*) AS orders FROM orders GROUP BY order_status",
  },
  {
    "question": "What is the average number of installments by payment type?",
    "sql": "SELECT payment_type, ROUND(AVG(payment_installments), 2) AS avg_installments FROM orders GROUP BY payment_type",
  },
  {
    "question": "Which 10 cities have the most unique customers?",
    "sql": "SELECT customer_city, COUNT(DISTINCT customer_unique_id) AS unique_customers FROM orders GROUP BY customer_city ORDER BY unique_customers DESC LIMIT 10",
  },
  {
    "question": "What was revenue in the furniture decor category in 2018?",
    "sql": "SELECT ROUND(SUM(oi.price), 2) AS revenue FROM order_items oi JOIN orders o ON oi.order_id = o.order_id WHERE o.order_purchase_timestamp <= '2018-12-31' AND oi.product_category_name_english = 'furniture_decor'",
  },
  {
    "question": "What is the distribution of review scores?",
    "sql": "SELECT review_score, COUNT(*) AS reviews FROM order_reviews GROUP BY review_score",
  },
  {
    "question": "What is the average order value by month in 2017?",
    "sql": "SELECT DATE_TRUNC('month', o.order_purchase_timestamp) AS month, AVG(o.order_value) AS avg_order_value FROM orders o WHERE o.order_purchase_timestamp <= '2017-12-31' GROUP BY month",
  },
  {
    "question": "Which sellers in RJ have the highest revenue?",
    "sql": "SELECT s.seller_id, ROUND(SUM(oi.price), 2) AS revenue FROM orders o JOIN order_items oi ON o.order_id = oi.order_id JOIN sellers s ON o.seller_id = s.seller_id WHERE o.order_purchase_timestamp <= '2018-12-31' AND s.seller_state = 'RJ' GROUP BY s.seller_id ORDER BY revenue DESC LIMIT 10",
  },
  {
    "question": "How many unique customers ordered in each year?",
    "sql": "SELECT EXTRACT(YEAR FROM o.order_purchase_timestamp) AS year, COUNT(DISTINCT customer_unique_id) AS unique_customers FROM orders o GROUP BY year",
  }
]
```

⚠️ **Critical rule: never put evaluation questions in** `examples.json`. If the exact test question is in the examples, the LLM just copies the answer and your accuracy is fake. This is called **data leakage**, and interviewers *will* ask about it. The examples teach *patterns* (the delivered filter, COALESCE translation, the unique_id count) that transfer to different questions.

3.5 Build the collections

📁 **File:** `rag/ingest.py`

```
python

"""
Builds two Qdrant collections: docs (markdown knowledge) and examples (questions)

Run: python -m rag.ingest
Re-run whenever you edit files in data/knowledge/.
"""

import json
from pathlib import Path

from langchain_community.document_loaders import DirectoryLoader, TextLoader
from langchain_core.documents import Document
from langchain_openai import OpenAIEmbeddings
from langchain_qdrant import QdrantVectorStore
from langchain_text_splitters import RecursiveCharacterTextSplitter

from config import settings


def build_docs_collection(embeddings):
    loader = DirectoryLoader(
        str(settings.KNOWLEDGE_DIR),
        glob="**/*.md",
        loader_cls=TextLoader,  # plain text: no extra dependencies
        loader_kwargs={"encoding": "utf-8"},
    )
    documents = loader.load()

    # Clean metadata: a short file name, plus the table it describes (if applicable)
    # The 'table' field lets us filter searches later.
    for doc in documents:
        name = Path(doc.metadata["source"]).stem
        doc.metadata["source"] = f"{name}.md"
        doc.metadata["table"] = name if name in settings.ALLOWED_TABLES else None
```

```

splitter = RecursiveCharacterTextSplitter(
    chunk_size=1200,
    chunk_overlap=150,
    separators=["\n## ", "\n\n", "\n", " "], # prefer splitting at
)
chunks = splitter.split_documents(documents)

QdrantVectorStore.from_documents(
    chunks,
    embeddings,
    url=settings.QDRANT_URL,
    collection_name=settings.DOCS_COLLECTION,
    force_recreate=True, # drop + rebuild, so re-runs no
)
print(f"{settings.DOCS_COLLECTION}: {len(documents)} files → {len(chunks)} chunks")

def build_examples_collection(embeddings):
    examples = json.loads((settings.KNOWLEDGE_DIR / "examples.json").read_text())
    documents = [
        Document(page_content=ex["question"], metadata={"sql": ex["sql"]})
        for ex in examples
    ]
    QdrantVectorStore.from_documents(
        documents,
        embeddings,
        url=settings.QDRANT_URL,
        collection_name=settings.EXAMPLES_COLLECTION,
        force_recreate=True,
    )
    print(f"{settings.EXAMPLES_COLLECTION}: {len(documents)} examples")

if __name__ == "__main__":
    embeddings = OpenAIEmbeddings(model=settings.EMBEDDING_MODEL)
    build_docs_collection(embeddings)
    build_examples_collection(embeddings)

```

What's happening:

- **Loading:** `TextLoader` reads each `.md` file. The default `DirectoryLoader` loader needs the heavy `unstructured` package, which we avoid.
- **Metadata:** Each document gets a clean `source` file name and a `table` field (e.g. `orders.md` → `table = "orders"`). This metadata is stored in Qdrant as the **payload** of each point.
- **Chunking:** Documents are split into ~1,200-character pieces. The

separators say "split at `##` headings first", so each chunk tends to be one coherent section (e.g. one metric). `chunk_overlap` repeats 150 characters between neighbours, so a sentence cut in half still appears whole in one chunk.

- **Embedding + upload:** `from_documents` embeds every chunk with OpenAI, creates the collection in Qdrant with the right vector size, and uploads each chunk as a point (vector + payload). `force_recreate=True` deletes the old collection first.
- **Examples collection:** We embed only the *question*, and keep the SQL in the payload. We want to find *similar questions*, then show their SQL.

💡 **Why is chunk size a real decision?** Chunks that are too big mean retrieved text is mostly irrelevant, which wastes tokens and distracts the LLM. Chunks that are too small mean a rule gets separated from the table it applies to. Our docs are short and header-structured, so section-sized chunks work well. This is a good thing to experiment with and report.

```
bash
```

```
python -m rag.build_value_docs
python -m rag.ingest
```

Look inside the vector database. Open <http://localhost:6333/dashboard>:

1. You should see two collections, `olist_docs` and `olist_examples`.
2. Click `olist_docs` and browse a few points. Each has an id, a vector of ~1,536 numbers, and a payload with `page_content` and `metadata`.
3. Screenshot the collection view for your README.

3.6 The retriever

📁 **File:** `rag/retriever.py`

```
python

"""
Search functions used by evaluation, the RAG MCP server and (indirectly)
"""

from functools import lru_cache

from langchain_openai import OpenAIEmbeddings
from langchain_qdrant import QdrantVectorStore
from qdrant_client import models

from config import settings
```

```

@lru_cache
def _store(collection: str) -> QdrantVectorStore:
    """Connect to an existing collection (created by rag/ingest.py)."""
    return QdrantVectorStore.from_existing_collection(
        embedding=OpenAIEmbeddings(model=settings.EMBEDDING_MODEL),
        collection_name=collection,
        url=settings.QDRANT_URL,
    )

def search_docs(query: str, k: int = 5, table: str | None = None) -> list:
    """Semantic search over docs. Optionally restrict to chunks about one
    qdrant_filter = None
    if table:
        qdrant_filter = models.Filter(must=[
            models.FieldCondition(key="metadata.table", match=models.MatchValue(value=table))
        ])

    hits = _store(settings.DOCS_COLLECTION).similarity_search_with_score(
        query, k=k, filter=qdrant_filter
    )
    return [
        {
            "source": doc.metadata.get("source", ""),
            "content": doc.page_content,
            "score": round(float(score), 4),      # cosine similarity: HIGH
        }
        for doc, score in hits
    ]

def search_examples(question: str, k: int = 3) -> list[dict]:
    hits = _store(settings.EXAMPLES_COLLECTION).similarity_search(question, k=k)
    return [{"question": d.page_content, "sql": d.metadata["sql"]} for d in hits]

def format_docs(docs: list[dict]) -> str:
    return "\n\n".join(f"[{d['source']}] {d['content']}" for d in docs)

def format_examples(examples: list[dict]) -> str:
    return "\n\n".join(f"Q: {e['question']} SQL: {e['sql']}" for e in examples)

if __name__ == "__main__":

```

```

for d in search_docs("How should revenue be calculated?"):
    print(d["source"], d["score"])
print()
for d in search_docs("which columns hold dates?", table="orders"):
    print("[filtered]", d["source"], d["score"])
print()
for e in search_examples("top categories by sales"):
    print(e["question"])

```

What's happening:

- `from_existing_collection` connects to a collection that `ingest.py` already built. `@lru_cache` keeps one connection per collection instead of reconnecting on every search.
- `similarity_search_with_score` embeds the query and asks Qdrant for the `(k)` nearest points. Qdrant returns **cosine similarity**, where *higher* means more similar (1.0 = identical direction).
- The optional **filter** narrows the search before ranking. `metadata.table = 'orders'` means "only consider chunks from `orders.md`". LangChain stores our metadata under a `metadata` key in the payload, which is why the filter key is `metadata.table`. This combination of vector search plus structured filtering is a core vector-database feature that a plain library index doesn't give you.
- We return plain dicts instead of LangChain objects, because in Phase 6 these results must travel through MCP as JSON.

 **Interview angle:** "Why cosine similarity?" OpenAI embeddings are normalized, so cosine similarity measures the angle between vectors: how similar the *meaning* is, independent of text length.

```
bash
```

```
python -m rag.retriever
```

Check that "How should revenue be calculated?" retrieves `metrics.md` or `order_items.md` near the top, and that the filtered search only returns `orders.md`. If the ranking is poor, improve the docs, not the code.

Phase 3 checkpoint (Milestone 2)

```
bash
```

```
python -m evaluation.evaluate_sql --mode rag
```

Compare against the baseline. Look especially at the questions tagged `trap` and `business_rule`. Then create a results table in your README:

markdown

```
| Experiment | Execution accuracy |  
|---|---|  
| A. Baseline (schema only) | xx% |  
| B. + RAG (docs + examples) | xx% |
```

bash

```
git add . && git commit -m "Phase 3: RAG with docs, value docs and few-shots"
```

Phase 4 — The Knowledge Graph

Goal: A graph of tables, columns, joins, metrics and business concepts in Neo4j. It gives the agent **join paths**, **metric definitions** and a **pruned schema**, with only the tables a question needs.

4.1 Why a graph, when RAG already has docs?

RAG answers "*what does this mean?*" by finding similar *text*. It cannot reliably answer "*what is the shortest valid way to connect* `customers` *to* `category_translation`?" That's a **path-finding problem**, and graphs solve it exactly:

text

```
customers -JOINS_TO- orders -JOINS_TO- order_items -JOINS_TO- products -SQL-
```

The graph also acts as a **semantic layer**: "sales" → the `revenue` metric → its exact SQL expression and filter → the tables it needs.

4.2 The graph model

text

```
(:Table {name, description})  
  | HAS_COLUMN  
  ▼  
(:Column {table, name, data_type})  
  
(:Table)-[:JOINS_TO {condition, kind: 'foreign_key' | 'soft', note}]->(:Table)  
  
(:Metric {name, synonyms, definition, sql, filter})-[:COMPUTED_FROM]->(:Table)  
  
(:Concept {name, synonyms})-[:STORED_IN]->(:Table)
```

Node	Comes from	Example
Table, Column	Automatically from Postgres	orders, orders.order_status
JOINS_TO (foreign_key)	Automatically from foreign keys	orders.customer_id = customers.customer_id
JOINS_TO (soft)	Hand-written in catalog.py	products ↔ category_translation
Metric	Hand-written in catalog.py	revenue, GMV, late delivery rate
Concept	Hand-written in catalog.py	"category", "location", "satisfaction"

 **The key design idea:** Structure is extracted *automatically*, and meaning is *curated* by a human. This mirrors how real data platforms (dbt semantic layer, data catalogs) work, and it's a strong talking point.

4.3 The semantic catalog

 **File:** graph/catalog.py

```
python
```

```
"""
Human-curated knowledge that cannot be derived from the database schema.
Everything structural (tables, columns, foreign keys) is read from Postgres
"""

TABLE_DESCRIPTIONS = {
    "orders": "One row per order: status and purchase/delivery timestamps",
    "order_items": "One row per item in an order: price (revenue), freight",
    "order_payments": "One row per payment of an order: payment type, inst",
    "order_reviews": "One row per review: score 1-5 and optional comment.",
    "customers": "Customer per order. customer_unique_id identifies the re",
    "sellers": "One row per seller with location (state, city, zip).",
    "products": "One row per product with Portuguese category name and dir",
    "category_translation": "Maps Portuguese category names to English.",
    "geolocation": "Lat/lng samples per zip prefix. Many rows per zip; ag",
}

# Real relationships that are not foreign keys in the database
SOFT_JOINS = [
```

```

    {
      "from": "products", "to": "category_translation",
      "condition": "products.product_category_name = category_translation.category_name",
      "note": "Use LEFT JOIN: some categories have no translation.",
    },
    {
      "from": "customers", "to": "geolocation",
      "condition": "customers.customer_zip_code_prefix = geolocation.geolocation_zip_code_prefix",
      "note": "Many geolocation rows per zip: aggregate geolocation by zip_code_prefix",
    },
    {
      "from": "sellers", "to": "geolocation",
      "condition": "sellers.seller_zip_code_prefix = geolocation.geolocation_zip_code_prefix",
      "note": "Many geolocation rows per zip: aggregate geolocation by zip_code_prefix",
    },
  ]

```

```

METRICS = [
  {
    "name": "revenue",
    "synonyms": ["revenue", "sales", "turnover", "income", "earned", "gross sales"],
    "definition": "Sum of item prices for delivered orders, excluding shipping costs.",
    "sql": "SUM(order_items.price)",
    "filter": "orders.order_status = 'delivered'",
    "tables": ["order_items", "orders"],
  },
  {
    "name": "gmv",
    "synonyms": ["gmv", "gross merchandise", "including freight", "income"],
    "definition": "Item price plus freight for delivered orders.",
    "sql": "SUM(order_items.price + order_items.freight_value)",
    "filter": "orders.order_status = 'delivered'",
    "tables": ["order_items", "orders"],
  },
  {
    "name": "average_order_value",
    "synonyms": ["average order value", "aov", "basket size", "average basket value"],
    "definition": "Revenue divided by number of distinct delivered orders.",
    "sql": "SUM(order_items.price) / COUNT(DISTINCT orders.order_id)",
    "filter": "orders.order_status = 'delivered'",
    "tables": ["order_items", "orders"],
  },
  {
    "name": "customer_count",
    "synonyms": ["customers", "buyers", "clients", "shoppers"],
    "definition": "Distinct real people. customer_id is per order, so each order has one customer.",
  }
]

```

```

        "sql": "COUNT(DISTINCT customers.customer_unique_id)",
        "filter": "",
        "tables": ["customers"],
    },
    {
        "name": "average_review_score",
        "synonyms": ["review", "rating", "satisfaction", "score", "stars"],
        "definition": "Average of review_score (1-5).",
        "sql": "ROUND(AVG(order_reviews.review_score), 2)",
        "filter": "",
        "tables": ["order_reviews"],
    },
    {
        "name": "delivery_time_days",
        "synonyms": ["delivery time", "shipping time", "how long", "days to delivery"],
        "definition": "Days from purchase to delivery for delivered orders.",
        "sql": "EXTRACT(EPOCH FROM (orders.order_delivered_customer_date - orders.order_purchase_timestamp))",
        "filter": "orders.order_status = 'delivered' AND orders.order_delivered_customer_date >= orders.order_purchase_timestamp",
        "tables": ["orders"],
    },
    {
        "name": "late_delivery_rate",
        "synonyms": ["late", "delayed", "on time", "on-time", "overdue"],
        "definition": "Share of delivered orders delivered after the estimated delivery date.",
        "sql": "AVG(CASE WHEN orders.order_delivered_customer_date > orders.order_estimated_delivery_date THEN 1 ELSE 0)",
        "filter": "orders.order_status = 'delivered' AND orders.order_delivered_customer_date >= orders.order_purchase_timestamp",
        "tables": ["orders"],
    },
]

```

```

CONCEPTS = [
    {"name": "product category", "synonyms": ["category", "categories", "product category"], "tables": ["products", "category_translation"]},
    {"name": "seller", "synonyms": ["seller", "merchant", "vendor", "store"], "tables": ["sellers", "order_items"]},
    {"name": "customer location", "synonyms": ["state", "city", "region", "location"], "tables": ["customers"]},
    {"name": "payment", "synonyms": ["payment", "paid", "credit card", "bill"], "tables": ["order_payments"]},
    {"name": "coordinates", "synonyms": ["latitude", "longitude", "distance"], "tables": ["geolocation"]},
    {"name": "product", "synonyms": ["product", "item", "weight", "photos"], "tables": ["products", "order_items"]},
]

```

 **What's happening:** This file is the **semantic layer**. It encodes the knowledge a human analyst carries in their head: "sales means revenue, revenue means `SUM(price)` on delivered orders, and category needs two tables."

4.4 Build the graph

 **File:** `graph/build_graph.py`

```
python
```

```
"""
```

```
Builds the Neo4j knowledge graph:
```

- structure (tables, columns, foreign keys) from Postgres automatically
- meaning (soft joins, metrics, concepts) from graph/catalog.py

```
Run: python -m graph.build_graph
```

```
"""
```

```
from functools import lru_cache
```

```
from neo4j import GraphDatabase
```

```
from config import settings
```

```
from database.introspect import get_columns, get_foreign_keys
```

```
from graph.catalog import CONCEPTS, METRICS, SOFT_JOINS, TABLE_DESCRIPTIONS
```

```
@lru_cache
```

```
def get_driver():
```

```
    return GraphDatabase.driver(
        settings.NEO4J_URI,
        auth=(settings.NEO4J_USERNAME, settings.NEO4J_PASSWORD),
    )
```

```
def build():
```

```
    columns = get_columns()
    fks = get_foreign_keys()
```

```
    tables = [{"name": t, "description": TABLE_DESCRIPTIONS.get(t, "")} for t in tables]
    cols = [{"table": t, "name": c, "type": dt} for t, cs in columns.items() for c, dt in cs]
```

```
    with get_driver().session() as session:
```

```
        # 0. Start clean so re-running never duplicates
        session.run("MATCH (n) DETACH DELETE n")
```

```
        # 1. Tables
```

```

session.run("""
    UNWIND $tables AS t
    MERGE (x:Table {name: t.name})
    SET x.description = t.description
""", tables=tables)

# 2. Columns
session.run("""
    UNWIND $cols AS c
    MATCH (t:Table {name: c.table})
    MERGE (col:Column {table: c.table, name: c.name})
    SET col.data_type = c.type
    MERGE (t)-[:HAS_COLUMN]->(col)
""", cols=cols)

# 3. Foreign-key joins (automatic)
session.run("""
    UNWIND $fks AS fk
    MATCH (a:Table {name: fk.table_name}), (b:Table {name: fk.foreign_table_name})
    MERGE (a)-[r:JOINS_TO {condition:
        fk.table_name + '.' + fk.column_name + ' = ' +
        fk.foreign_table_name + '.' + fk.foreign_column_name}]->(b)
    SET r.kind = 'foreign_key', r.note = ''
""", fks=fks)

# 4. Soft joins (curated)
session.run("""
    UNWIND $joins AS j
    MATCH (a:Table {name: j.from}), (b:Table {name: j.to})
    MERGE (a)-[r:JOINS_TO {condition: j.condition}]->(b)
    SET r.kind = 'soft', r.note = j.note
""", joins=SOFT_JOINS)

# 5. Metrics
session.run("""
    UNWIND $metrics AS m
    MERGE (x:Metric {name: m.name})
    SET x.synonyms = m.synonyms, x.definition = m.definition,
        x.sql = m.sql, x.filter = m.filter
    WITH x, m
    UNWIND m.tables AS table_name
    MATCH (t:Table {name: table_name})
    MERGE (x)-[:COMPUTED_FROM]->(t)
""", metrics=METRICS)

# 6. Concepts

```

```

session.run("""
    UNWIND $concepts AS c
    MERGE (x:Concept {name: c.name})
    SET x.synonyms = c.synonyms
    WITH x, c
    UNWIND c.tables AS table_name
    MATCH (t:Table {name: table_name})
    MERGE (x)-[:STORED_IN]->(t)
""", concepts=CONCEPTS)

print(f"Graph built: {len(tables)} tables, {len(cols)} columns, "
      f"{len(fks)} FK joins, {len(SOFT_JOINS)} soft joins, "
      f"{len(METRICS)} metrics, {len(CONCEPTS)} concepts")

if __name__ == "__main__":
    build()

```



What's happening, the Cypher basics:

- `MERGE` means "find this pattern, or create it if it doesn't exist." It's like an upsert.
- `UNWIND $list AS x` loops over a Python list inside one query. This is far faster than sending one query per item.
- `$tables` is a **parameter**. Values are sent separately from the query text, just like parameterized SQL, which prevents injection.
- `(a)-[r:JOINS_TO]->(b)` is a relationship pattern. The arrow direction records which table holds the foreign key.

bash

```
python -m graph.build_graph
```

Now look at your graph. Open <http://localhost:7474> and run:

cypher

```
MATCH (t:Table)-[r:JOINS_TO]-(u:Table) RETURN t, r, u
```

cypher

```
MATCH (m:Metric)-[:COMPUTED_FROM]->(t:Table) RETURN m, t
```



Take a screenshot of the first result for your README. A picture of the schema graph instantly communicates what this component does.

4.5 Graph queries

 **File:** graph/queries.py

```
python
```

```
"""
```

```
Questions we ask the Knowledge Graph.
```

```
"""
```

```
from itertools import combinations
```

```
from graph.build_graph import get_driver
```

```
JOIN_PATH_QUERY = """
```

```
MATCH (a:Table {name: $a}), (b:Table {name: $b})
```

```
MATCH p = shortestPath((a)-[:JOINS_TO*..6]-(b))
```

```
RETURN [r IN relationships(p) |
```

```
        {condition: r.condition, kind: r.kind, note: r.note}] AS steps
```

```
"""
```

```
METRICS_QUERY = """
```

```
MATCH (m:Metric)
```

```
WHERE any(s IN m.synonyms WHERE toLower($q) CONTAINS s)
```

```
OPTIONAL MATCH (m)-[:COMPUTED_FROM]->(t:Table)
```

```
RETURN m.name AS name, m.definition AS definition, m.sql AS sql,
```

```
        m.filter AS filter, collect(t.name) AS tables
```

```
"""
```

```
CONCEPTS_QUERY = """
```

```
MATCH (c:Concept)
```

```
WHERE any(s IN c.synonyms WHERE toLower($q) CONTAINS s)
```

```
MATCH (c)-[:STORED_IN]->(t:Table)
```

```
RETURN c.name AS name, collect(t.name) AS tables
```

```
"""
```

```
TABLE_DETAILS_QUERY = """
```

```
MATCH (t:Table)-[:HAS_COLUMN]->(c:Column)
```

```
WHERE t.name IN $tables
```

```
RETURN t.name AS table, t.description AS description,
```

```
        collect(c.name + ' ' + c.data_type) AS columns
```

```
"""
```

```
def _run(query: str, **params) -> list[dict]:
```

```
    with get_driver().session() as session:
```

```
        return [record.data() for record in session.run(query, **params)]
```

```

def find_metrics(question: str) -> list[dict]:
    return _run(METRICS_QUERY, q=question)

def find_concepts(question: str) -> list[dict]:
    return _run(CONCEPTS_QUERY, q=question)

def find_join_paths(tables: list[str]) -> list[dict]:
    """Union of shortest join paths between every pair of tables."""
    seen, steps = set(), []
    for a, b in combinations(sorted(set(tables)), 2):
        for record in _run(JOIN_PATH_QUERY, a=a, b=b):
            for step in record["steps"]:
                if step["condition"] not in seen:
                    seen.add(step["condition"])
                    steps.append(step)
    return steps

def get_table_details(tables: list[str]) -> list[dict]:
    return _run(TABLE_DETAILS_QUERY, tables=list(tables))

def build_graph_context(question: str, candidate_tables: list[str] | None) -> dict:
    """
    Combine everything the graph knows about a question into prompt-ready
    Returns {"tables": [...], "context": "..."}
    """
    metrics = find_metrics(question)
    concepts = find_concepts(question)

    tables = set(candidate_tables or [])
    for item in metrics + concepts:
        tables.update(item["tables"])
    if not tables:
        tables = {"orders"}  # sensible hub default

    joins = find_join_paths(list(tables))
    # Join paths can pass through tables we didn't list (e.g. orders) - in
    for step in joins:
        left, right = step["condition"].split(" = ")
        tables.add(left.split(".")[0])
        tables.add(right.split(".")[0])

```

```

lines = ["Relevant tables:"]
for t in get_table_details(sorted(tables)):
    lines.append(f"- {t['table']}: {t['description']}")

if joins:
    lines.append("\nJoin path:")
    for s in joins:
        note = f"  -- {s['note']}" if s["note"] else ""
        lines.append(f"- {s['condition']}{note}")

if metrics:
    lines.append("\nMetric definitions:")
    for m in metrics:
        where = f" WHERE {m['filter']}" if m["filter"] else ""
        lines.append(f"- {m['name']}: {m['sql']}{where}. {m['definition']}")

return {"tables": sorted(tables), "context": "\n".join(lines)}

if __name__ == "__main__":
    result = build_graph_context("revenue by category for customers in SP")
    print(result["tables"])
    print(result["context"])

```

What's happening:

1. **Metric and concept matching:** The question's words are matched against synonyms. "revenue by category" matches the `revenue` metric (→ `order_items`, `orders`) and the `product category` concept (→ `products`, `category_translation`). "SP"/"state" matches `customer location` (→ `customers`).
2. **Join paths:** For every pair of those tables, Neo4j's `shortestPath` finds the fewest hops through `JOINS_TO` edges. `-[:JOINS_TO*..6]-` means "1 to 6 hops, either direction."
3. **Pass-through tables:** If the path to `customers` goes through `orders`, we add `orders` too. Otherwise the LLM would get a join condition referring to a table it wasn't told about.
4. **Formatting:** Everything is turned into a short text block for the prompt.

 **Why this beats "just send the whole schema":** With 9 tables the whole schema fits in a prompt anyway, so the gain here is mostly *correct joins* (especially the soft-join warnings). But this design scales. With 300 tables the full schema won't fit, and *graph-based schema pruning* becomes essential. Mention this scalability argument in interviews.

⚠️ **Honest limitation:** Synonym matching is simple substring matching ("state" also matches "statement"). A good future improvement is embedding-based concept matching. The agent in Phase 6 also passes LLM-picked `candidate_tables`, which covers the gaps.

```
bash
```

```
python -m graph.queries
```

✅ Phase 4 checkpoint (Milestone 3)

```
bash
```

```
python -m evaluation.evaluate_sql --mode rag_kg
```

Add row C to your README table. Look especially at questions tagged `multi_join`.

```
bash
```

```
git add . && git commit -m "Phase 4: Neo4j knowledge graph with join paths"
```

Phase 5 — SQL Safety Validation

Goal: Before any LLM-written SQL touches the database, verify that it's a single read-only SELECT on allowed tables, and add a row limit.

5.1 The validator

📁 **File:** `database/sql_validator.py`

```
python

"""
Parses SQL with sqlglot and enforces safety rules.
This is layer 2 of our defense; layer 1 is the read-only database user.
"""

from dataclasses import dataclass

import sqlglot
from sqlglot import exp

from config import settings

# Statement types that must never appear anywhere in the tree.
# getattr(...) because class names vary slightly across sqlglot versions.
```

```

_FORBIDDEN_NAMES = ["Insert", "Update", "Delete", "Drop", "Create", "Alter",
                    "AlterTable", "Merge", "TruncateTable", "Command", "Grant"]
FORBIDDEN_NODES = tuple(
    node for node in (getattr(exp, name, None) for name in _FORBIDDEN_NAMES)
    if node is not None
)

QUERY_NODES = (exp.Select, exp.Union, exp.Intersect, exp.Except)

@dataclass
class ValidationResult:
    valid: bool
    sql: str = ""  # the (possibly rewritten) safe SQL
    error: str = ""

def validate_sql(sql: str, max_rows: int = settings.MAX_ROWS) -> ValidationResult:
    # 1. Must parse
    try:
        statements = [s for s in sqlglot.parse(sql, read="postgres") if s]
    except sqlglot.errors.ParseError as exc:
        return ValidationResult(False, error=f"SQL syntax error: {str(exc)}")

    # 2. Exactly one statement ("SELECT 1; DROP TABLE x" is two)
    if len(statements) != 1:
        return ValidationResult(False, error=f"Expected exactly 1 statement")
    stmt = statements[0]

    # 3. Must be a query
    if not isinstance(stmt, QUERY_NODES):
        return ValidationResult(False, error=f"Only SELECT queries are allowed")

    # 4. No write operations hidden anywhere (e.g. inside a CTE)
    bad = next(stmt.find_all(*FORBIDDEN_NODES), None)
    if bad is not None:
        return ValidationResult(False, error=f"Forbidden operation: {bad}")

    # 5. SELECT ... INTO creates a table
    if stmt.args.get("into"):
        return ValidationResult(False, error="SELECT INTO is not allowed.")

    # 6. Only known tables (CTE names are allowed, they're temporary)
    cte_names = {cte.alias_or_name for cte in stmt.find_all(exp.CTE)}
    for table in stmt.find_all(exp.Table):
        if table.name and table.name not in cte_names and table.name not in settings.KNOWN_TABLES:
            return ValidationResult(False, error=f"Table '{table.name}' is not known")

```

```
# 7. Enforce a row limit if the query has none
if stmt.args.get("limit") is None:
    stmt = stmt.limit(max_rows)

return ValidationResult(True, sql=stmt.sql(dialect="postgres"))
```

 **What's happening:** `sqlglot` turns the SQL string into an **abstract syntax tree (AST)**, a tree of Python objects (`Select`, `Join`, `Table`, `Where`...). Checking the *tree* is far more reliable than searching the *text*. Consider this query:

```
sql

SELECT 'please DROP this' AS note FROM orders
```

A text search for "DROP" would wrongly reject it. The AST correctly sees just a `Select` with a string literal. And this query:

```
sql

WITH x AS (DELETE FROM orders RETURNING *) SELECT * FROM x
```

is a `SELECT` at the top, but has a `Delete` node hidden inside. `find_all` walks the whole tree and catches it.

 **Why a validator when the DB user is read-only anyway?**

1. **Better error messages for self-correction.** "Table 'sales' does not exist" is clearer to the LLM than a raw Postgres error.
2. **The row limit.** A read-only user can still `SELECT *` from 1M geolocation rows and crash your UI.
3. **The allowed-table policy.** It blocks reading system tables like `pg_user`.
4. **Defense in depth.** If someone misconfigures the DB user, the validator still protects the data.

5.2 Tests

 **File:** `tests/test_sql_validator.py`

```
python
```

```

import pytest

from database.sql_validator import validate_sql

@pytest.mark.parametrize("sql", [
    "SELECT COUNT(*) FROM orders",
    "SELECT o.order_id FROM orders o JOIN customers c ON c.customer_id = o.customer_id",
    "WITH x AS (SELECT * FROM orders) SELECT COUNT(*) FROM x",
    "SELECT 'please DROP this' AS note FROM orders",
])
def test_valid_queries(sql):
    assert validate_sql(sql).valid

@pytest.mark.parametrize("sql", [
    "DROP TABLE orders",
    "DELETE FROM orders",
    "UPDATE orders SET order_status = 'x'",
    "INSERT INTO orders (order_id) VALUES ('x')",
    "SELECT 1; DROP TABLE orders",
    "SELECT * INTO new_table FROM orders",
    "SELECT * FROM pg_user",
    "SELECT * FROM secret_table",
    "SELEC * FORM orders",
])
def test_rejected_queries(sql):
    assert not validate_sql(sql).valid

def test_limit_added_when_missing():
    result = validate_sql("SELECT * FROM orders")
    assert "LIMIT 1000" in result.sql

def test_existing_limit_kept():
    result = validate_sql("SELECT * FROM orders LIMIT 5")
    assert "LIMIT 5" in result.sql and "1000" not in result.sql

```

Phase 5 checkpoint

bash

```
pytest tests/test_sql_validator.py -v
```



If a test fails because of your sqlglot version, read how sqlglot parsed it: `print(repr(sqlglot.parse_one("...")))`. Understanding the AST is part of the learning.

bash

```
git add . && git commit -m "Phase 5: sqlglot SQL validator with tests"
```

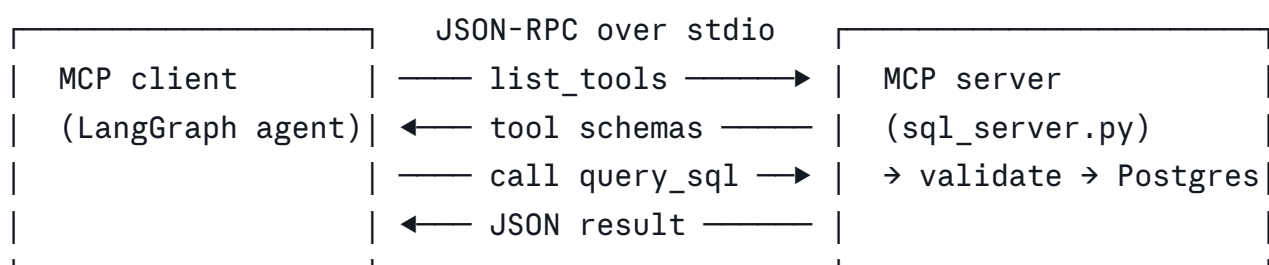
Phase 6 — MCP (Model Context Protocol)

Goal: Expose SQL, RAG, the Knowledge Graph and analytics as **MCP servers**. The agent will reach every capability *only* through MCP tools.

6.1 What MCP is

MCP is an open protocol (like USB for AI tools). A **server** declares tools with names, descriptions and typed inputs. Any **client** (your LangGraph agent, Claude Desktop, an IDE) can discover and call them.

text



stdio transport: the client starts the server as a subprocess and they talk through stdin/stdout.



Why MCP instead of just importing Python functions?

1. **A security boundary.** The agent can *only* do what the tools allow. There's no `query_graph(cypher)` tool with arbitrary Cypher. Tools are narrow and parameterized.
2. **Reusability.** The same SQL server works in Claude Desktop or any MCP client with zero changes. That makes a great demo.
3. **Decoupling.** You could rewrite the RAG server with a different vector DB, and the agent wouldn't change.



The #1 MCP bug with stdio: Never `print()` inside a stdio server. stdout is the protocol channel, so a stray print corrupts the messages and the client hangs or crashes. Use `logging` to stderr instead.

6.2 Analytics helper (used by the analytics server)

python

```
import pandas as pd

def summarize(df: pd.DataFrame, max_preview: int = 10) -> dict:
    """Compact facts about a result, for the LLM to write an answer from."""
    summary = {
        "row_count": len(df),
        "columns": list(df.columns),
        "preview": df.head(max_preview).to_dict(orient="records"),
        "numeric": {},
    }
    for col in df.select_dtypes("number").columns:
        s = df[col].dropna()
        if len(s):
            summary["numeric"][col] = {
                "min": float(s.min()), "max": float(s.max()),
                "mean": round(float(s.mean()), 2), "sum": round(float(s.sum()), 2)
            }
    # For 2-column "label, value" results, precompute the top and bottom
    if len(df.columns) == 2 and len(summary["numeric"]) == 1 and len(df) > 10:
        label, value = df.columns[0], list(summary["numeric"])[0]
        ordered = df.sort_values(value, ascending=False)
        summary["top"] = ordered.iloc[0].to_dict()
        summary["bottom"] = ordered.iloc[-1].to_dict()
    return summary
```



Why compute stats in Python instead of asking the LLM? LLMs are unreliable at arithmetic over many rows. Pandas computes the sums and means *exactly*, and the LLM only *describes* the verified numbers. This is how you prevent hallucinated figures.

6.3 SQL server

 **File:** mcp_servers/sql_server.py

python

```
"""
MCP server: read-only access to the Olist Postgres database.
Run standalone for debugging: python -m mcp_servers.sql_server
"""

import json
```

```

import logging
import sys

from mcp.server.fastmcp import FastMCP

from database.executor import run_select
from database.introspect import get_schema_text
from database.sql_validator import validate_sql

logging.basicConfig(stream=sys.stderr, level=logging.INFO) # NEVER log
log = logging.getLogger("sql_server")

mcp = FastMCP("olist-sql")

@mcp.tool()
def get_schema.tables: list[str] | None = None) -> str:
    """Return the database schema (columns and foreign keys).
    Pass a list of table names to get only those tables."""
    return get_schema_text(tables)

@mcp.tool()
def query_sql(sql: str) -> str:
    """Execute a single read-only PostgreSQL SELECT query on the Olist database.
    Returns JSON with columns and rows, or an error message."""
    check = validate_sql(sql)
    if not check.valid:
        log.info("rejected: %s", check.error)
        return json.dumps({"ok": False, "error": check.error, "sql": sql})

    try:
        df = run_select(check.sql)
    except Exception as exc:
        error = str(exc).strip().splitlines()[0]
        log.info("db error: %s", error)
        return json.dumps({"ok": False, "error": error, "sql": check.sql})

    return json.dumps({
        "ok": True,
        "sql": check.sql,
        "columns": list(df.columns),
        "rows": json.loads(df.to_json(orient="records", date_format="iso")),
        "row_count": len(df),
    })

```

```
if __name__ == "__main__":  
    mcp.run()    # stdio transport by default
```



What's happening:

- `FastMCP` handles the whole protocol. You only write normal functions.
- `@mcp.tool()` registers a function as a tool. The **function name** becomes the tool name, the **docstring** becomes the description the agent reads, and the **type hints** become the input schema. Write docstrings as instructions for an AI, because that's who reads them.
- The server validates the SQL *again*, even though the agent validates too. Servers should never trust clients.
- Results go out as a JSON string. `default_handler=str` guarantees that any odd type (dates, decimals) can be serialized.

6.4 RAG server



File: `mcp_servers/rag_server.py`

python

```

"""MCP server: semantic search over Olist documentation and solved examples"""
import json
import logging
import sys

from mcp.server.fastmcp import FastMCP

from rag.retriever import search_docs as _search_docs
from rag.retriever import search_examples as _search_examples

logging.basicConfig(stream=sys.stderr, level=logging.INFO)
mcp = FastMCP("olist-rag")

@mcp.tool()
def search_docs(query: str, k: int = 5, table: str | None = None) -> str:
    """Search table documentation, business rules, metric definitions,
    data-quality notes and real column values in the vector database.
    Optionally pass a table name to search only that table's documentation.
    Returns JSON list of chunks with similarity scores."""
    return json.dumps(_search_docs(query, k, table))

@mcp.tool()
def search_examples(question: str, k: int = 3) -> str:
    """Find previously solved questions similar to this one, with their SQL solutions.
    Returns JSON list of {question, sql}."""
    return json.dumps(_search_examples(question, k))

if __name__ == "__main__":
    mcp.run()

```

6.5 Graph server

 **File:** `mcp_servers/graph_server.py`

python

```

"""MCP server: the Olist schema knowledge graph (Neo4j)."""
import json
import logging
import sys

from mcp.server.fastmcp import FastMCP

from graph import queries

logging.basicConfig(stream=sys.stderr, level=logging.INFO)
mcp = FastMCP("olist-graph")

@mcp.tool()
def build_graph_context(question: str, candidate_tables: list[str] | None):
    """Given a question (and optionally tables you think are relevant), return relevant tables, the join path between them and matching metric definitions. Returns JSON {tables, context}."""
    return json.dumps(queries.build_graph_context(question, candidate_tables))

@mcp.tool()
def find_join_path(tables: list[str]) -> str:
    """Return the join conditions needed to connect the given tables. JSON"""
    return json.dumps(queries.find_join_paths(tables))

@mcp.tool()
def find_metrics(question: str) -> str:
    """Return business metric definitions (SQL expression + filter) mentioning the given metric name"""
    return json.dumps(queries.find_metrics(question))

if __name__ == "__main__":
    mcp.run()

```



Notice what's missing: There's no `run_cypher(query)` tool. Giving an LLM a raw query language on your graph would reopen every injection risk we closed for SQL. Narrow tools = small attack surface.

6.6 Analytics server

 **File:** `mcp_servers/analytics_server.py`

```
python
```

```

"""MCP server: deterministic analysis of query results with pandas."""
import json
import logging
import sys

import pandas as pd
from mcp.server.fastmcp import FastMCP

from analytics.analysis import summarize

logging.basicConfig(stream=sys.stderr, level=logging.INFO)
mcp = FastMCP("olist-analytics")

@mcp.tool()
def analyze_result(rows: list[dict]) -> str:
    """Compute exact statistics (row count, min/max/mean/sum, top/bottom
    over query result rows. Use these numbers instead of doing arithmetic
    return json.dumps(summarize(pd.DataFrame(rows)), default=str)

if __name__ == "__main__":
    mcp.run()

```

6.7 Test the servers by hand

 **File:** `scripts/test_mcp_client.py`

```
python
```

```

"""
Starts the SQL server as a subprocess, lists its tools and calls one.
Run: python -m scripts.test_mcp_client
"""

import asyncio
import json
import sys

from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client

from config import settings

async def main():
    params = StdioServerParameters(
        command=sys.executable,
        args=["-m", "mcp_servers.sql_server"],
        cwd=str(settings.ROOT),
    )
    async with stdio_client(params) as (read, write):
        async with ClientSession(read, write) as session:
            await session.initialize()

            tools = await session.list_tools()
            for tool in tools.tools:
                print(f"TOOL {tool.name}: {tool.description.splitlines()}")

            result = await session.call_tool(
                "query_sql", {"sql": "SELECT order_status, COUNT(*) FROM orders"}
            )
            print(json.dumps(json.loads(result.content[0].text), indent=2))

            blocked = await session.call_tool("query_sql", {"sql": "DROP TABLE orders"})
            print("\nDROP attempt →", blocked.content[0].text)

if __name__ == "__main__":
    asyncio.run(main())

```



What's happening: This is the raw MCP protocol, with no LangChain involved. `stdio_client` launches the server subprocess, and `initialize` performs the handshake. `list_tools` shows exactly what an agent would see, and `call_tool` sends a request. `cwd=ROOT` makes the subprocess start in the project root, so its imports and `.env` loading work.

```
bash

python -m scripts.test_mcp_client
```

Optional: the official visual MCP Inspector (requires Node.js):

```
bash

npx @modelcontextprotocol/inspector python -m mcp_servers.sql_server
```

It opens a browser UI where you can click through the tools and call them manually. That's excellent for debugging, and it makes a nice screenshot for your README.

✓ Phase 6 checkpoint

- `test_mcp_client` lists 2 tools, returns order-status counts, and rejects the DROP.
- Adapt the script (change `args`) to test `rag_server`, `graph_server` and `analytics_server` too.

```
bash

git add . && git commit -m "Phase 6: MCP servers for SQL, RAG, graph and analytics"
```

Phase 7 — The LangGraph Agent

Goal: A stateful workflow that understands the question, gathers context through MCP, generates and validates SQL, **self-corrects on errors**, asks for clarification when needed, and remembers the conversation.

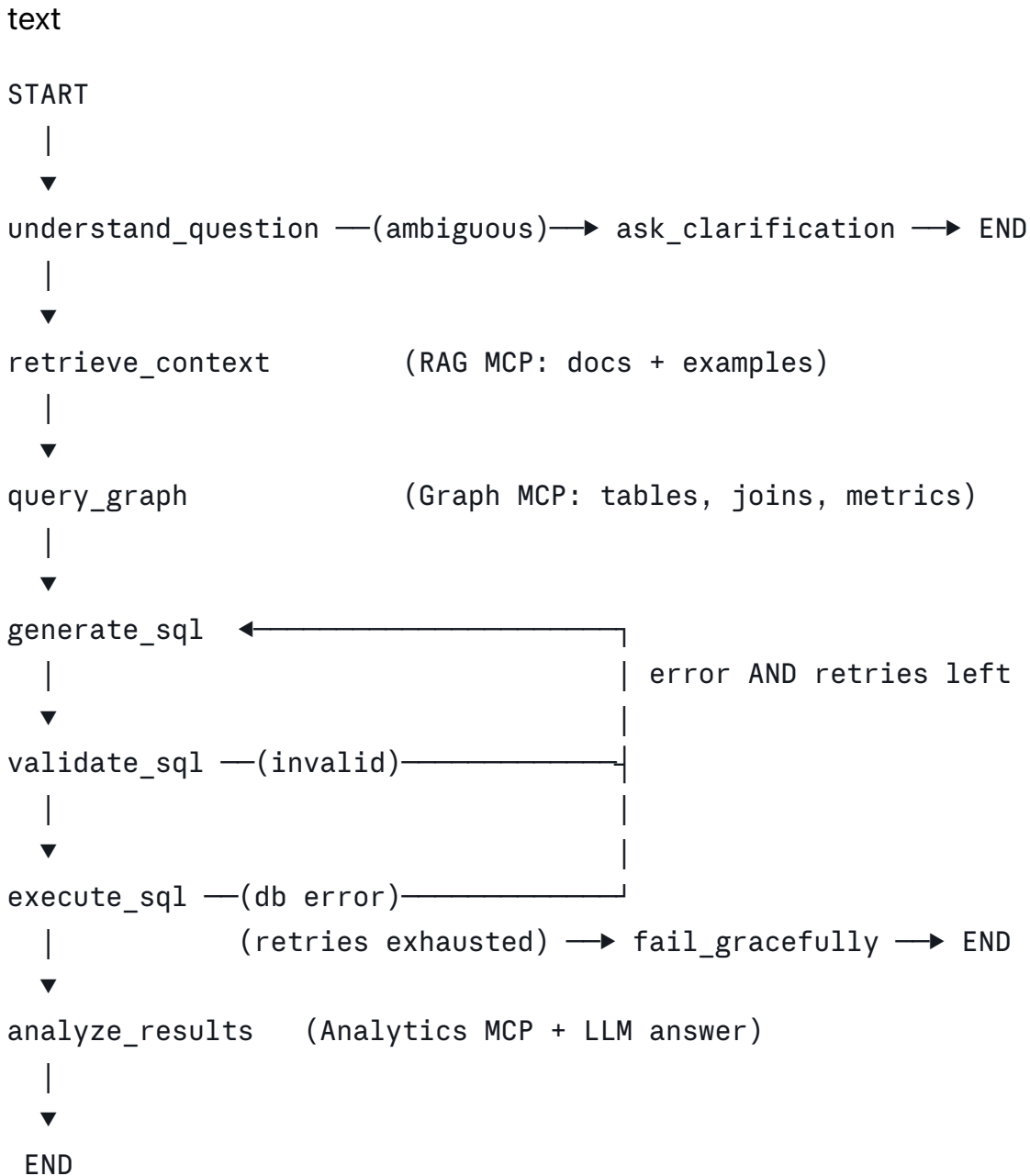
7.1 LangGraph concepts in 60 seconds

Concept	Meaning	In our agent
State	A dict shared by every step	question, sql, error, rows, answer...
Node	A function: <code>state</code> → <code>partial update</code>	<code>generate_sql</code> , <code>execute_sql</code> ...
Edge	"after A, go to B"	<code>retrieve_context</code> → <code>query_graph</code>
Conditional edge	A function that <i>chooses</i> the next node	"SQL failed and retries remain → back to <code>generate_sql</code> "
Checkpoint	Saves state between invocations	Conversation memory per <code>thread_id</code>

💡 **Why LangGraph and not a simple function chain?** Because of the **loop**.

"Generate → validate → execute → (error) → generate again with the error" is a cycle with a counter and multiple exits. In plain Python this becomes nested ifs and whiles. In LangGraph it's an explicit, visualizable graph where each step is traced separately.

7.2 The workflow



7.3 State

📁 **File:** `agent/state.py`

python

```

from typing import TypedDict

class AgentState(TypedDict, total=False):
    # --- input ---
    question: str

    # --- understanding ---
    standalone_question: str      # follow-ups rewritten to stand alone
    candidate_tables: list[str]   # LLM's guess at relevant tables
    needs_clarification: bool
    clarification: str

    # --- context ---
    rag_context: str
    examples: str
    graph_context: str
    relevant_tables: list[str]    # tables the KG says are needed

    # --- SQL loop ---
    sql: str
    sql_error: str
    retry_count: int

    # --- results ---
    columns: list[str]
    rows: list[dict]
    analysis: dict
    final_answer: str

    # --- memory + observability ---
    history: list[dict]           # previous turns: question, sql, answer
    trace: list[str]              # human-readable log of steps in this turn

```



What's happening: `TypedDict` documents the shape of the state.

`total=False` means not every key must exist at every moment. Each node returns only the keys it changes, and LangGraph merges them into the state.

7.4 MCP client wrapper

File: `agent/tools.py`

```
python
```

```
"""
```

```
Connects the agent to our four MCP servers and exposes a simple call_tool
```

```

"""
import json
import sys

from langchain_mcp_adapters.client import MultiServerMCPCClient

from config import settings

def _stdio(module: str) -> dict:
    return {
        "command": sys.executable,          # the venv's python
        "args": ["-m", module],
        "transport": "stdio",
        "cwd": str(settings.ROOT),          # so imports and .env work in
    }

SERVERS = {
    "sql": _stdio("mcp_servers.sql_server"),
    "rag": _stdio("mcp_servers.rag_server"),
    "graph": _stdio("mcp_servers.graph_server"),
    "analytics": _stdio("mcp_servers.analytics_server"),
}

_client = MultiServerMCPCClient(SERVERS)
_tools: dict | None = None

async def get_tools() -> dict:
    """Discover tools from all servers once, then cache them by name."""
    global _tools
    if _tools is None:
        _tools = {tool.name: tool for tool in await _client.get_tools()}
    return _tools

def _to_text(result) -> str:
    """Adapter versions return either a string or a list of content blocks
    if isinstance(result, str):
        return result
    if isinstance(result, list):
        parts = []
        for block in result:
            if isinstance(block, dict):
                parts.append(block.get("text", ""))

```

```

        else:
            parts.append(getattr(block, "text", str(block)))
        return "".join(parts)
    return str(result)

```

```

async def call_tool(name: str, **arguments):
    """Call an MCP tool by name and parse its JSON output."""
    tools = await get_tools()
    raw = _to_text(await tools[name].ainvoke(arguments))
    try:
        return json.loads(raw)
    except json.JSONDecodeError:
        return raw

```



What's happening: `MultiServerMCPClient` manages all four servers.

`get_tools()` performs MCP tool discovery and converts every MCP tool into a LangChain tool object. We index them by name, so nodes can call `call_tool("query_sql", sql=...)`.



Two design choices worth knowing:

- **Deterministic tool calls:** Our nodes decide *which* tool to call, rather than letting the LLM freely choose. For a data system that needs reliability, a fixed workflow with LLM-powered steps beats a free-roaming agent. It's more predictable, testable and cheaper. (Section 13 lists a "free tool choice" variant as an upgrade.)
- **Performance:** By default each tool call starts a short-lived server session. That's fine for learning. A later optimization is keeping persistent sessions open.

7.5 Nodes



File: `agent/nodes.py`

```

python

"""
Each node takes the current state and returns a partial update.
"""

import json

from langchain_core.messages import HumanMessage, SystemMessage
from pydantic import BaseModel, Field

from agent.tools import call_tool
from config import settings
from database.sql_validator import validate_sql

```

```
from graph.catalog import TABLE_DESCRIPTIONS
from llm.client import get_llm
from llm.sql_generator import generate_sql


def _trace(state, message: str) -> list[str]:
    return state.get("trace", []) + [message]


def _history_text(state, turns: int = 3) -> str:
    history = state.get("history", [])[-turns:]
    return "\n".join(f>User: {h['question']}\nSQL: {h['sql']}" for h in history)


# ----- 1. understand question -----

class QuestionAnalysis(BaseModel):
    standalone_question: str = Field(
        description="The question rewritten to be fully self-contained, using only information from the provided context."
    )
    candidate_tables: list[str] = Field(
        description="Tables likely needed, chosen only from the provided list of available tables."
    )
    needs_clarification: bool = Field(
        description="True ONLY if the question cannot be answered without clarification."
    )
    clarification_question: str = Field(
        default="", description="What to ask the user, if clarification is needed."
    )


UNDERSTAND_PROMPT = """You analyse questions about the Olist Brazilian e-commerce database (orders from Sep 2016 to Oct 2018).

Available tables:
{tables}

Conversation so far:
{history}

New user message: {question}

1. Rewrite the message as a standalone question (resolve "it", "that", "what").
2. Pick candidate tables from the list.
3. Ask for clarification ONLY if the question is truly unanswerable as-is (e.g. "show me the thing"). Do not ask for clarification if a reasonable answer exists: assume calendar quarters, delivered orders for revenue, and all dates are in YYYY-MM format unless no year is given."""


async def understand_question(state):
```

```

tables = "\n".join(f"- {t}: {d}" for t, d in TABLE_DESCRIPTIONS.items)
llm = get_llm().with_structured_output(QuestionAnalysis)
analysis: QuestionAnalysis = await llm.ainvoke(UNDERSTAND_PROMPT.format(
    tables=tables, history=_history_text(state), question=state["ques

candidates = [t for t in analysis.candidate_tables if t in settings.AL
return {
    "standalone_question": analysis.standalone_question,
    "candidate_tables": candidates,
    "needs_clarification": analysis.needs_clarification,
    "clarification": analysis.clarification_question,
    "trace": _trace(state, f"understand → '{analysis.standalone_quest
}

```

```

async def ask_clarification(state):
    return {
        "final_answer": state["clarification"],
        "trace": _trace(state, "clarification requested"),
    }

```

```

# ----- 2. context retrieval -----
async def retrieve_context(state):
    q = state["standalone_question"]
    docs = await call_tool("search_docs", query=q, k=5)
    examples = await call_tool("search_examples", question=q, k=3)

    return {
        "rag_context": "\n\n".join(f"[{d['source']}] \n {d['content']}" for
        "examples": "\n\n".join(f"Q: {e['question']} \n SQL: {e['sql']}" for
        "trace": _trace(state, f"RAG → {[d['source'] for d in docs]}"),
    }

```

```

async def query_graph(state):
    result = await call_tool(
        "build_graph_context",
        question=state["standalone_question"],
        candidate_tables=state.get("candidate_tables", []),
    )
    return {
        "graph_context": result["context"],
        "relevant_tables": result["tables"],
        "trace": _trace(state, f"KG → tables {result['tables']}"),
    }

```

```

# ----- 3. SQL

async def generate_sql_node(state):
    retry = state.get("retry_count", 0)

    # First attempt: pruned schema (KG tables). On retries: full schema,
    tables = state.get("relevant_tables") if retry == 0 else None
    schema = await call_tool("get_schema", tables=tables)

    error_feedback = ""
    if state.get("sql_error"):
        error_feedback = f"SQL:\n{state['sql']}\n\nError:\n{state['sql_error']}"

    sql = generate_sql(
        state["standalone_question"],
        schema,
        rag_context=state.get("rag_context", ""),
        examples=state.get("examples", ""),
        graph_context=state.get("graph_context", ""),
        error_feedback=error_feedback,
    )
    label = "generate SQL" if retry == 0 else f"regenerate SQL (retry {retry})"
    return {"sql": sql, "trace": _trace(state, label)}

async def validate_sql_node(state):
    result = validate_sql(state["sql"])
    if result.valid:
        return {"sql": result.sql, "sql_error": "", "trace": _trace(state, "valid")}
    return {
        "sql_error": result.error,
        "retry_count": state.get("retry_count", 0) + 1,
        "trace": _trace(state, f"validation failed: {result.error}"),
    }

async def execute_sql(state):
    result = await call_tool("query_sql", sql=state["sql"])
    if result.get("ok"):
        return {
            "columns": result["columns"],
            "rows": result["rows"],
            "sql_error": "",
            "trace": _trace(state, f"executed → {result['row_count']} rows"),
        }

```

```

    return {
        "sql_error": result["error"],
        "retry_count": state.get("retry_count", 0) + 1,
        "trace": _trace(state, f"execution failed: {result['error']}"),
    }

```

----- 4. answer

```
ANSWER_PROMPT = """Question: {question}
```

SQL used:

```
{sql}
```

Exact statistics computed from the result (trust these numbers):

```
{analysis}
```

Write a concise answer (2-5 sentences) for a business user.

- Use ONLY numbers that appear above. Never estimate or invent figures.
- Money is in BRL. Mention important caveats (e.g. only delivered orders).
- If the result is empty, say so and suggest why.

```
async def analyze_results(state):
```

```
    analysis = await call_tool("analyze_result", rows=state.get("rows", []))
    response = await get_llm().ainvoke([
```

```
        SystemMessage(content="You are a precise data analyst."),
```

```
        HumanMessage(content=ANSWER_PROMPT.format(
```

```
            question=state["standalone_question"],
```

```
            sql=state["sql"],
```

```
            analysis=json.dumps(analysis, indent=2, default=str)[:6000],
```

```
        ]),
```

```
    ])
```

```
    turn = {"question": state["standalone_question"], "sql": state["sql"]}
```

```
    return {
```

```
        "analysis": analysis,
```

```
        "final_answer": response.content,
```

```
        "history": state.get("history", []) + [turn],
```

```
        "trace": _trace(state, "analysis + answer"),
```

```
    }
```

```
async def fail_gracefully(state):
```

```
    message = (
```

```
        "I couldn't produce a working query for this question after "
```

```
        f"{state.get('retry_count', 0)} attempts.\n\n"
```

```
        f"Last error: {state.get('sql_error')}\n\nLast SQL tried:\n{state.get('sql_error')}
```

```
"Try rephrasing, or name the metric and time period explicitly."
)
return {"final_answer": message, "trace": _trace(state, "gave up after
```

What's happening, node by node:

- `understand_question` uses **structured output**.
`with_structured_output(QuestionAnalysis)` forces the LLM to return JSON matching the Pydantic model, so we get a typed object instead of free text we'd have to parse. This node also resolves follow-ups using `history`.
- `retrieve_context` and `query_graph` call MCP tools. Note they use the *standalone* question, so "what about Q4?" retrieves Q4-relevant context.
- `generate_sql_node` implements a smart retry strategy. The first attempt uses the KG's **pruned schema**. If it fails, the retry gets the **full schema** plus the **error message**. This handles the case where the graph missed a needed table.
- `validate_sql_node` and `execute_sql` increment `retry_count` on failure and store the error. They don't decide what happens next. That's the edges' job.
- `analyze_results` gets exact stats from the analytics MCP server, then the LLM writes the answer *only from those numbers*.

 **Why nodes don't decide routing:** Separation of concerns. Nodes *do work*, and edges *make decisions*. That makes the routing logic trivially unit-testable (see Phase 10).

7.6 Edges

 **File:** `agent/edges.py`

```
python
```

```

"""
Routing functions. Pure functions of state → next node name. Easy to test
"""

from config import settings


def route_after_understanding(state) -> str:
    return "ask_clarification" if state.get("needs_clarification") else "

def _retry_or_fail(state) -> str:
    return "generate_sql" if state.get("retry_count", 0) < settings.MAX_RE

def route_after_validation(state) -> str:
    return "execute_sql" if not state.get("sql_error") else _retry_or_fai

def route_after_execution(state) -> str:
    return "analyze_results" if not state.get("sql_error") else _retry_or

```

7.7 Wiring the graph

 **File:** `agent/graph.py`

```

python

from langgraph.checkpoint.memory import MemorySaver
from langgraph.graph import END, START, StateGraph

from agent import nodes
from agent.edges import route_after_execution, route_after_understanding,
from agent.state import AgentState


def build_graph(checkpointer=None):
    g = StateGraph(AgentState)

    g.add_node("understand_question", nodes.understand_question)
    g.add_node("ask_clarification", nodes.ask_clarification)
    g.add_node("retrieve_context", nodes.retrieve_context)
    g.add_node("query_graph", nodes.query_graph)
    g.add_node("generate_sql", nodes.generate_sql_node)
    g.add_node("validate_sql", nodes.validate_sql_node)
    g.add_node("execute_sql", nodes.execute_sql)

```

```

g.add_node("analyze_results", nodes.analyze_results)
g.add_node("fail_gracefully", nodes.fail_gracefully)

g.add_edge(START, "understand_question")
g.add_conditional_edges("understand_question", route_after_understand:
                        ["ask_clarification", "retrieve_context"])
g.add_edge("ask_clarification", END)
g.add_edge("retrieve_context", "query_graph")
g.add_edge("query_graph", "generate_sql")
g.add_edge("generate_sql", "validate_sql")
g.add_conditional_edges("validate_sql", route_after_validation,
                        ["execute_sql", "generate_sql", "fail_gracefully"])
g.add_conditional_edges("execute_sql", route_after_execution,
                        ["analyze_results", "generate_sql", "fail_gracefully"])
g.add_edge("analyze_results", END)
g.add_edge("fail_gracefully", END)

return g.compile(checkpointer=checkpointer or MemorySaver())

def new_turn(question: str) -> dict:
    """Input for one user message. Resets per-turn fields; history persists"""
    return {
        "question": question,
        "sql": "",
        "sql_error": "",
        "retry_count": 0,
        "rows": [],
        "columns": [],
        "analysis": {},
        "final_answer": "",
        "needs_clarification": False,
        "trace": [],
    }

if __name__ == "__main__":
    print(build_graph().get_graph().draw_mermaid())

```



What's happening:

- `add_conditional_edges(source, router, [possible targets])` means "after `source`, call `router(state)` and go to whichever node name it returns."
- `MemorySaver` is the **checkpointer**. After each run, the full state is saved under a `thread_id`. The next message with the same `thread_id` starts

from the saved state, which is why `history` survives between questions.

- `new_turn` resets per-question fields (the retry counter, trace, SQL) but deliberately **doesn't** include `history`, so the saved history is kept.

💡 **Visualize your graph:** `python -m agent.graph` prints Mermaid code. Paste it into <https://mermaid.live> and put the image in your README.

⚠️ `MemorySaver` keeps memory in RAM only, so it's lost on restart. For persistence you'd switch to a SQLite or Postgres checkpointer. That's a good "future work" item.

7.8 CLI to chat with the agent

📁 **File:** `scripts/run_agent.py`

```
python
```

```

"""
Chat with the full agent in the terminal.
Run: python -m scripts.run_agent
"""

import asyncio
import uuid

from agent.graph import build_graph, new_turn

async def main():
    graph = build_graph()
    config = {"configurable": {"thread_id": str(uuid.uuid4())}}
    print("Ask about the Olist data (empty line to quit).\n")

    while True:
        question = input("You: ").strip()
        if not question:
            break
        state = await graph.ainvoke(new_turn(question), config=config)

        print("\n--- trace ---")
        for step in state.get("trace", []):
            print("  •", step)
        if state.get("sql"):
            print("\n--- SQL ---\n", state["sql"])
        print("\nCopilot:", state["final_answer"], "\n")

if __name__ == "__main__":
    asyncio.run(main())

```

✓ Phase 7 checkpoint (Milestone 4)

bash

python -m scripts.run_agent

Try this sequence and **read the trace each time**:

text

You: What was total revenue in 2017?	
You: What about 2018?	← tests memory/rewriting
You: Top 5 categories by revenue	
You: How many customers are from RJ?	← tests the unique_id trap
You: Show me the thing	← should ask for clarification

To see **self-correction** happen, temporarily break the SQL prompt (e.g. add "use the column total_price" to `SQL_SYSTEM_PROMPT`). Then watch the trace show `execution failed → regenerate SQL (retry 1) → executed`. Remove the break afterwards.

```
bash
```

```
git add . && git commit -m "Phase 7: LangGraph agent with self-correction"
```

Phase 8 — Visualization + Streamlit UI

8.1 Chart selection

 **File:** `analytics/visualization.py`

```
python
```

```
"""
Pick a sensible chart for a result, or None if a chart wouldn't help.
"""
import pandas as pd
import plotly.express as px

def _as_datetime(series: pd.Series) -> pd.Series | None:
    if pd.api.types.is_datetime64_any_dtype(series):
        return series
    if series.dtype == object:
        converted = pd.to_datetime(series, errors="coerce", utc=False)
        if converted.notna().all():
            return converted
    return None

def make_chart(df: pd.DataFrame):
    if df.empty or len(df) < 2 or len(df.columns) < 2:
        return None  # single values don't

    numeric = df.select_dtypes("number").columns.tolist()
    if not numeric:
```

```

        return None
    y = numeric[0]

    # Time series → line chart
    for col in df.columns:
        if col == y:
            continue
        dates = _as_datetime(df[col])
        if dates is not None:
            plot_df = df.assign(**{col: dates}).sort_values(col)
            return px.line(plot_df, x=col, y=y, markers=True)

    # Category + value → bar chart (only if readable)
    labels = [c for c in df.columns if c not in numeric]
    if labels and len(df) <= 30:
        return px.bar(df, x=labels[0], y=y)

    # Two numbers → scatter
    if len(numeric) >= 2:
        return px.scatter(df, x=numeric[0], y=numeric[1])

    return None

```

 **What's happening:** This is a rule-based policy. Dates plus a number make a line chart, a label plus a number makes a bar chart, and two numbers make a scatter plot. Dates come back from MCP as ISO strings, so we try converting them.

 **Why rules instead of asking the LLM to pick a chart?** They're deterministic, free, instant and testable. An LLM chart picker is a possible upgrade, but only once rules prove insufficient.

8.2 The Streamlit app

 **File:** `app/main.py`

```

python

"""
Run: streamlit run app/main.py
"""

import asyncio
import sys
import uuid
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent)) # make

```

```

import pandas as pd
import streamlit as st

from agent.graph import build_graph, new_turn
from analytics.visualization import make_chart

st.set_page_config(page_title="Olist Analytics Copilot", page_icon="📊",
st.title("📊 Olist Analytics Copilot")
st.caption("Ask questions about ~100k Brazilian e-commerce orders (2016-2018)")

@st.cache_resource
def get_graph():
    return build_graph() # built once; MemorySaver lives inside st

if "thread_id" not in st.session_state:
    st.session_state.thread_id = str(uuid.uuid4())
    st.session_state.messages = []

with st.sidebar:
    st.subheader("Try asking")
    for example in [
        "Top 5 categories by revenue",
        "Monthly revenue in 2018",
        "What about 2017?",
        "Do late deliveries get worse reviews?",
        "How many repeat customers are there?",
    ]:
        st.code(example, language=None)
    if st.button("New conversation"):
        st.session_state.thread_id = str(uuid.uuid4())
        st.session_state.messages = []
        st.rerun()

def render_result(msg: dict):
    st.markdown(msg["answer"])
    if msg.get("sql"):
        with st.expander("SQL"):
            st.code(msg["sql"], language="sql")
    if msg.get("rows"):
        df = pd.DataFrame(msg["rows"], columns=msg.get("columns"))
        with st.expander(f"Data ({len(df)} rows)":
            st.dataframe(df, use_container_width=True)

```

```

        chart = make_chart(df)
        if chart is not None:
            st.plotly_chart(chart, use_container_width=True)
    with st.expander("Agent trace"):
        for step in msg.get("trace", []):
            st.markdown(f"- {step}")

# Replay the conversation
for msg in st.session_state.messages:
    with st.chat_message(msg["role"]):
        if msg["role"] == "user":
            st.markdown(msg["content"])
        else:
            render_result(msg)

# New question
if question := st.chat_input("Ask a question about your data..."):
    st.session_state.messages.append({"role": "user", "content": question})
    with st.chat_message("user"):
        st.markdown(question)

    with st.chat_message("assistant"):
        with st.spinner("Thinking..."):
            config = {"configurable": {"thread_id": st.session_state.thread_id}}
            state = asyncio.run(get_graph()).ainvoke(new_turn(question), config)
            msg = {
                "role": "assistant",
                "answer": state["final_answer"],
                "sql": state.get("sql"),
                "rows": state.get("rows"),
                "columns": state.get("columns"),
                "trace": state.get("trace", []),
            }
        render_result(msg)
    st.session_state.messages.append(msg)

```

What's happening:

- Streamlit **re-runs the whole script** on every interaction. So `st.session_state` holds the chat messages and thread id, and `@st.cache_resource` keeps one graph (and its memory) alive across reruns.
- The graph is async, so `asyncio.run` executes one full agent run per question.
- The **Agent trace** expander is your proof that the system is genuinely

agentic. It shows the RAG sources, KG tables and any retries.

```
bash
```

```
streamlit run app/main.py
```

✅ Phase 8 checkpoint (Milestone 5)

Record a **2-minute screen recording** now: a simple question, a follow-up, a chart and an expanded trace. You'll reuse it for your resume and LinkedIn.

```
bash
```

```
git add . && git commit -m "Phase 8: Streamlit UI with charts and agent t
```

Phase 9 — Evaluation: The Full Experiment

9.1 Evaluate the full agent

📄 **File:** `evaluation/evaluate_agent.py`

```
python
```

```
"""
```

```
Experiment D: the full LangGraph + MCP agent.
```

```
Run: python -m evaluation.evaluate_agent
```

```
"""
```

```
import asyncio
```

```
import json
```

```
import time
```

```
import uuid
```

```
import pandas as pd
```

```
from agent.graph import build_graph, new_turn
```

```
from config import settings
```

```
from database.executor import run_select
```

```
from evaluation.compare import results_match
```

```
EVAL_DIR = settings.ROOT / "evaluation"
```

```
async def evaluate():
```

```
    graph = build_graph()
```

```
    questions = json.loads((EVAL_DIR / "questions.json").read_text())
```

```
    results = []
```

```

for item in questions:
    start = time.time()
    config = {"configurable": {"thread_id": str(uuid.uuid4())}} # f
    state = await graph.ainvoke(new_turn(item["question"]), config=config)

    predicted = pd.DataFrame(state.get("rows") or [], columns=state.get("columns"))
    gold = run_select(item["gold_sql"])
    correct = bool(state.get("rows")) and results_match(predicted, gold)

    results.append({
        "id": item["id"],
        "question": item["question"],
        "tags": item.get("tags", []),
        "sql": state.get("sql"),
        "correct": correct,
        "retries": state.get("retry_count", 0),
        "clarified": bool(state.get("needs_clarification")),
        "latency_s": round(time.time() - start, 2),
        "trace": state.get("trace", []),
    })
    print(f"{'✅' if correct else '❌'} {item['id']} retries={results.get('retries', 0)}")

n = len(results)
report = {
    "mode": "agent",
    "accuracy": sum(r["correct"] for r in results) / n,
    "questions_needing_retry": sum(r["retries"] > 0 for r in results),
    "recovered_after_retry": sum(r["retries"] > 0 and r["correct"] for r in results),
    "avg_latency_s": round(sum(r["latency_s"] for r in results) / n, 2),
    "results": results,
}

(EVAL_DIR / "results" / "agent.json").write_text(json.dumps(report, indent=2))
print(json.dumps({k: v for k, v in report.items() if k != "results"}, indent=2))

if __name__ == "__main__":
    asyncio.run(evaluate())

```



What's happening: The same questions and the same comparison function are used, now through the whole agent. We also measure **agentic** metrics: how often a retry was needed, and how often the retry *fixed* the problem. Note that rounding in the agent's SQL can make an otherwise-correct result mismatch, and `compare.py` rounds floats to 2 decimals to absorb most of that.

9.2 The results table

```
bash
```

```
python -m evaluation.evaluate_sql --mode baseline
python -m evaluation.evaluate_sql --mode rag
python -m evaluation.evaluate_sql --mode rag_kg
python -m evaluation.evaluate_agent
```

Put this in your README (with *your real numbers*):

```
markdown
```

```
| Experiment | Components | Exec. accuracy | Trap questions | Avg latency |
|---|---|---|---|---|
| A | LLM + schema | xx% | x/4 | x.xs |
| B | + RAG (docs, values, examples) | xx% | x/4 | x.xs |
| C | + Knowledge Graph | xx% | x/4 | x.xs |
| D | Full agent (LangGraph + MCP + self-correction) | xx% | x/4 | x.xs |
```



How to read your results honestly:

- Run each experiment **2–3 times**. LLM output varies a little even at temperature 0. Report the average.
- If the KG doesn't improve accuracy on 9 tables, **say so** and explain the scalability argument. Honest negative results impress senior engineers far more than suspiciously perfect ones.
- Look at *which tags* each component fixed. For example: *"RAG fixed business-rule errors; the KG fixed multi-join errors; self-correction recovered 3 of 4 runtime failures."* That sentence is gold in an interview.

9.3 Optional: RAG retrieval quality

For each eval question, write down which knowledge file *should* be retrieved, then measure how often it appears in the top 5 (**hit rate@5**). This isolates retrieval quality from SQL generation. If you want standard metrics (context precision, faithfulness), look into the **RAGAS** library.

```
bash
```

```
git add . && git commit -m "Phase 9: full evaluation A-D"
```

Phase 10 — Tests, Observability, Docker

10.1 Routing tests

 **File:** `tests/test_edges.py`

python

```
from agent.edges import route_after_execution, route_after_understanding,
from config import settings

def test_clarification_route():
    assert route_after_understanding({"needs_clarification": True}) == "ask_for_clarification"
    assert route_after_understanding({"needs_clarification": False}) == "route_to_validation"

def test_valid_sql_goes_to_execution():
    assert route_after_validation({"sql_error": ""}) == "execute_sql"

def test_error_with_retries_left_regenerates():
    assert route_after_execution({"sql_error": "boom", "retry_count": 1}) == "route_to_validation"

def test_error_after_max_retries_fails_gracefully():
    state = {"sql_error": "boom", "retry_count": settings.MAX_RETRIES}
    assert route_after_execution(state) == "fail_gracefully"
    assert route_after_validation(state) == "fail_gracefully"

def test_success_goes_to_analysis():
    assert route_after_execution({"sql_error": "", "retry_count": 2}) == "route_to_analysis"
```

bash

```
pytest -v
```



What we test and why: The validator (security), the comparison (evaluation correctness) and the routing (agent control flow) are **deterministic**, so they get fast unit tests with no LLM or database. LLM behaviour is measured by the *evaluation set*, not by unit tests. Knowing this distinction is a sign of AI-engineering maturity.

10.2 Observability with LangSmith

Add to `.env`:

text

```
LANGSMITH_TRACING=true
LANGSMITH_API_KEY=your-langsmith-key
LANGSMITH_PROJECT=olist-copilot
```

That's it. LangChain and LangGraph send traces automatically. At <https://smith.langchain.com> you'll see every run as a tree: each node, each LLM call with its exact prompt, each tool call, the tokens and the latency.

💡 **Use it to debug:** When the agent gets a question wrong, open its trace and find the first step where things went off track. Was the wrong doc retrieved? Were the wrong tables picked? Was the right context ignored? That tells you which component to fix.

10.3 Dockerize the app

 **File:** `Dockerfile`

```
dockerfile

FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8501
CMD ["streamlit", "run", "app/main.py", "--server.address=0.0.0.0"]
```

Add the app service to `docker-compose.yml` (under `services:`):

yaml

```

app:
  build: .
  ports:
    - "8501:8501"
  env_file: .env
  environment:
    # inside Docker, databases are reached by service name, not localhost
    DATABASE_URL: postgresql+psycpg2://admin:admin@postgres:5432/olist
    READONLY_DATABASE_URL: postgresql+psycpg2://copilot_ro:readonly@pos
    NEO4J_URI: bolt://neo4j:7687
    QDRANT_URL: http://qdrant:6333
  depends_on:
    - postgres
    - neo4j
    - qdrant

```

 **What's happening:** Inside Docker's network, containers reach each other by service name (`postgres`, `neo4j`, `qdrant`). The `environment` values override `.env` for the container only. The embeddings already live in the Qdrant container's volume, so the app image doesn't need to contain or rebuild them.

bash

```

docker compose up -d --build
# open http://localhost:8501

```

bash

```

git add . && git commit -m "Phase 10: tests, LangSmith tracing, Dockerfile

```

Phase 11 (OPTIONAL) — LoRA Fine-Tuning

 **Only do this after Phases 0-10 are complete and measured.** It's not required for your four core technologies. It needs a GPU (a free Colab T4 works for a 1.5B model), and it only makes sense if you can compare it against Experiment C.

11.1 The idea

Instead of a large API model, fine-tune a small open model (e.g. `Qwen/Qwen2.5-Coder-1.5B-Instruct`, Apache-2.0 licensed) to generate SQL for *this* database. **LoRA** freezes the model's billions of weights and trains small low-rank "adapter" matrices (a few million parameters) inserted into the attention layers.

text

Frozen base model weights W (not trained)
+
Adapter: $B \times A$ (small, trained; rank $r = 16$)
=
Effective weights $W + BA$

11.2 Dataset

You need **200-500 question → SQL pairs** that are **not** in `evaluation/questions.json`. Build them by:

1. Writing ~50 by hand, covering every pattern (joins, CTEs, dates, CASE, ranking).
2. Using a strong LLM to write variations, then **executing every generated SQL** and discarding any that fail or look wrong.

 **File:** `finetuning/prepare_dataset.py`

python

```

"""
Builds finetuning/train.jsonl from finetuning/pairs.json ([{question, sql}]
Every SQL is executed; broken pairs are dropped. Eval questions are excluded
"""

import json

from config import settings
from database.executor import run_select
from database.introspect import get_schema_text

FT = settings.ROOT / "finetuning"

def main():
    pairs = json.loads((FT / "pairs.json").read_text())
    eval_questions = {q["question"].lower() for q in
                      json.loads((settings.ROOT / "evaluation/questions.json").read_text())}
    schema = get_schema_text()

    kept = 0
    with open(FT / "train.jsonl", "w") as out:
        for p in pairs:
            if p["question"].lower() in eval_questions:
                continue # no leakage
            try:
                run_select(p["sql"])
            except Exception:
                continue # drop broken SQL
            out.write(json.dumps({"question": p["question"], "schema": schema}))
            kept += 1
    print(f"kept {kept}/{len(pairs)} pairs")

if __name__ == "__main__":
    main()

```

11.3 Training

bash

```
pip install torch transformers datasets peft accelerate
```

 **File:** finetuning/train_lora.py

python

```
from datasets import load_dataset
from peft import LoraConfig, get_peft_model
from transformers import (AutoModelForCausalLM, AutoTokenizer, DataCollatorForSeq2Seq,
                          GradientDescentTrainer, TrainingArguments)

BASE = "Qwen/Qwen2.5-Coder-1.5B-Instruct"
tokenizer = AutoTokenizer.from_pretrained(BASE)
model = AutoModelForCausalLM.from_pretrained(BASE, torch_dtype="auto", device_map="auto")

lora = LoraConfig(
    r=16,                                # rank of the adapter matrices
    lora_alpha=32,                        # scaling factor (commonly 2 × r)
    lora_dropout=0.05,
    target_modules=["q_proj", "k_proj", "v_proj", "o_proj"],  # attention modules
    task_type="CAUSAL_LM",
)
model = get_peft_model(model, lora)
model.print_trainable_parameters()      # typically well under 1% of all parameters

def tokenize(example):
    messages = [
        {"role": "system", "content": "Write a PostgreSQL query. Output only the query."},
        {"role": "user", "content": f"{example['schema']}\n\nQuestion: {example['question']}\n\nAnswer:"}
    ]
    prompt = tokenizer.apply_chat_template(messages, tokenize=False, add_generation_prompt=True)
    prompt_ids = tokenizer(prompt)["input_ids"]
    full = tokenizer(prompt + example["sql"] + tokenizer.eos_token,
                     truncation=True, max_length=2048)

    # Only learn to produce the SQL: mask the prompt tokens with -100
    labels = [-100] * len(prompt_ids) + full["input_ids"][len(prompt_ids):]
    return {"input_ids": full["input_ids"], "attention_mask": full["attention_mask"],
            "labels": labels[:len(full["input_ids"])]}

data = load_dataset("json", data_files="finetuning/train.jsonl")["train"]
data = data.train_test_split(test_size=0.1, seed=42)
data = data.map(tokenize, remove_columns=data["train"].column_names)

trainer = GradientDescentTrainer(
    model=model,
    args=TrainingArguments(
        output_dir="finetuning/output",
    )
)
```

```

num_train_epochs=3,
per_device_train_batch_size=1,
gradient_accumulation_steps=8,
learning_rate=2e-4,
logging_steps=10,
eval_strategy="epoch",
save_strategy="epoch",
),
train_dataset=data["train"],
eval_dataset=data["test"],
data_collator=DataCollatorForSeq2Seq(tokenizer, label_pad_token_id=-100)
)
trainer.train()
model.save_pretrained("finetuning/output/adapters")

```



What's happening:

- **Label masking (-100):** The loss is computed only on the SQL tokens. The model learns to *write SQL given a question*, not to reproduce the schema.
- **r and alpha:** The rank controls the adapter's capacity, and alpha scales its influence. `r=16, alpha=32` is a common starting point.
- **Validation split:** This watches for overfitting. If the eval loss rises while the train loss falls, stop earlier.
- If you run out of GPU memory, use **QLoRA** (load the base model in 4-bit with `bitsandbytes`). Only do that for a real memory constraint.

11.4 Evaluate it fairly

Write a `generate_sql_local(question, schema, **context)` function that loads the base model plus the adapter with `PeftModel.from_pretrained`, and compare three things:

text

Small base model (no adapter) vs Small model + LoRA vs API model

A plausible honest outcome is that LoRA hugely improves the small model but still trails the API model, while being cheaper and running locally. That is a perfectly good, publishable result.

Part 12 — README, Resume, Interviews

12.1 README structure

Your README is the first thing a recruiter sees. Use this outline:

markdown

```

# Olist Analytics Copilot
One-line description + 20-second GIF of the UI.

## Results
The A-D experiment table.

## Architecture
Diagram + the one-sentence design principle.

## How it works
Short paragraph each: RAG + Qdrant vector DB (with dashboard screenshot),
MCP servers, LangGraph workflow (with Mermaid diagram).

## Safety
Read-only role, AST validator, row limits, timeouts, narrow MCP tools.

## Run it
docker compose up, kaggle download, load_data, build Qdrant collections,

## Lessons learned / limitations / future work

```

12.2 Resume entry

Only write what you've actually built and measured. Replace the X/Y with your numbers.

Olist Agentic Analytics Copilot | Python, LangGraph, MCP, RAG, Qdrant, Neo4j, PostgreSQL

- Built a natural-language analytics agent over a real 9-table e-commerce database (100k orders), raising SQL execution accuracy from **X% to Y%** over a schema-only baseline on a 40-question benchmark.
- Designed a Neo4j knowledge graph (auto-extracted from foreign keys, plus curated metrics) for join-path discovery and schema pruning, and a RAG layer on a Qdrant vector database (metadata-filtered semantic search) over business rules, column values and few-shot examples.
- Exposed SQL, retrieval, graph and analytics capabilities as 4 MCP servers, orchestrated by a stateful LangGraph workflow with clarification, conversational memory and error-driven self-correction (recovered **Z%** of failed queries).
- Enforced safety through a read-only DB role, AST-based SQL validation (sqlglot), row limits and query timeouts.

12.3 Questions you should be able to answer

Question	Where you learned it
Why not store the data in the vector DB?	Part 0.5
Why a vector database (Qdrant) instead of a library (FAISS)? What's a payload filter?	Phase 3.1, 3.6
What's the difference between what RAG gives and what the KG gives?	Phase 4.1
How do you prevent the LLM from deleting data?	Phases 0.4, 1.5, 5
Why validate the AST instead of searching the SQL text?	Phase 5.1
What is data leakage and how did you avoid it?	Phase 3.4
What is execution accuracy and what are its limits?	Phase 2.7
Why MCP instead of plain function calls?	Phase 6.1
Why a fixed workflow instead of a free-roaming agent?	Phase 7.4
How does self-correction work, and when does it give up?	Phases 7.5–7.6
How would this scale to 300 tables?	Phase 4.5 (schema pruning)
How do you stop the LLM hallucinating numbers?	Phase 6.2
Which component helped most, and how do you know?	Phase 9

Part 13 — Improvement Ideas (after everything works)

Pick these up once Milestone 5 is done and measured:

1. **Auto-onboarding for any database.** Point it at any Postgres URL, auto-generate the table docs with an LLM, auto-build the graph, and let users edit metric definitions in the UI.
2. **Embedding-based concept matching** in the KG instead of substring synonyms.
3. **Hybrid search in Qdrant.** Combine dense vectors with keyword (sparse/BM25) vectors, so exact terms like `boleto` or `customer_unique_id` rank well too.
4. **Parallel context retrieval.** Run `retrieve_context` and `query_graph` simultaneously (LangGraph fan-out).
5. **Human-in-the-loop.** Use LangGraph `interrupt` to show the SQL and ask for approval before running expensive queries.
6. **Persistent memory.** Swap `MemorySaver` for a Postgres checkpointer.

- 7. **Query decomposition** for multi-step questions ("compare A vs B, then explain why").
- 8. **Free tool choice.** A ReAct-style variant where the LLM chooses MCP tools itself, compared against the fixed workflow on your eval set.
- 9. **Use your MCP servers from Claude Desktop.** This is a great demo of reusability.
- 10. **A bigger benchmark.** Add BIRD-style harder questions and track accuracy per tag.
- 11. **Cost tracking.** Record tokens per query and report cost per correct answer.

Milestones Summary

Milestone	You have	Phase
1	Question → LLM → SQL → Postgres, with a measured baseline	2
2	+ RAG, measured	3
3	+ Knowledge Graph, measured	4
4	Full agent via MCP + LangGraph, working in the CLI	7
5	UI with charts and trace + demo video	8
6	Full evaluation, tests, tracing, Docker, README	9-10

At every milestone: it works, it's measured, it's committed.

The goal isn't merely to finish the app. By the end, you should be able to explain why every component exists, what it fixed (with numbers), how it fails, and what you'd do next. That's what makes this your No. 1 project.